

**LAGENGINE: ELIMINATING LAG: A REAL-TIME PHYSICS SIMULATION
ENGINE WITH TEMPLATE-DRIVEN CLASSROOM AUTHORIZING AND
ADAPTIVE RL-BASED RUNTIME OPTIMISATION**

by

SATRAJIT GHOSH

A thesis submitted to the
Graduate School of Rutgers, The State University of New Jersey
in partial fulfillment of the requirements
for the degree of Master of Science
Graduate Program in Electrical and Computer Engineering

Written under the direction of

Dr Dov Kruger

and approved by

New Brunswick, New Jersey

May 2026

ABSTRACT

Interactive physics demonstrations are widely used in science and engineering education; however, existing tools present a persistent trade-off between accessibility and performance. Full-scale game engines offer advanced rendering and physically accurate simulation but require substantial programming expertise and configuration effort, rendering them unsuitable for most classroom practitioners. Conversely, lightweight educational platforms often provide simplified interfaces at the expense of physical fidelity, extensibility, and reliable performance on heterogeneous, low-power hardware commonly found in instructional environments. This gap constrains educators who seek to deploy real-time, visually rich, and conceptually rigorous simulations without engaging in low-level engine development or manual performance tuning.

This thesis introduces *LAGEngine: Eliminating Lag: A Real-Time Physics Simulation Engine with Template-Driven Classroom Authoring and Adaptive RL-Based Runtime Optimisation*, a unified C++17-based 3D engine that integrates advanced simulation, rendering, authoring, and adaptive optimisation within a single architecture. The first layer consists of a custom-built engine core featuring rigid body dynamics, XPBD-based soft body and cloth simulation, SPH fluid systems, constraint solvers, deferred physically based rendering, an Entity-Component System with JSON serialisation, Lua scripting, robotics kinematics, and a multithreaded job system with GPU-driven submission. The second layer provides an educator-facing visual editor built with Dear ImGui, offering template-driven, low-code authoring workflows that expose high-level simulation parameters while abstracting engine complexity. The third layer embeds a passive reinforcement-learning auto-tuner that observes runtime telemetry and learns cross-session configuration policies to promote stable frame times across heterogeneous classroom hardware. Together, these contributions demonstrate that template-driven authoring, a multithreaded GPU-driven low-overhead rendering runtime, and an integrated RL-based optimisation layer can be cohesively combined to enable accessible yet high-fidelity physics simulation suitable for low-power educational settings.

Contents

1	Introduction	1
2	Background and Related Work	2
2.0.1	Interactive Physics Education Tools	2
2.0.2	Physics Simulation Methods	2
2.0.3	Real Time Rendering Architectures	2
2.0.4	Game Engine Architecture	3
2.0.5	Reinforcement Learning for System Optimisation	3
2.0.6	Synthesis and Identified Gap	3
3	Problem Statement and Design Goals	4
4	Prototype Evolution and Design Rationale	4
4.0.1	Phase 1: Verlet Cloth Prototypes	4
4.0.2	Phase 2: Robosim Robotics Prototype	4
4.0.3	Phase 3: Bullet Physics Evaluation and Rejection	5
4.0.4	Design Principles Derived from Prototyping	5
5	System Architecture Overview	5
5.1	Architecture Overview	5
5.2	Data Flow Across Layers	6
5.3	Deployment Model	6
6	Engine Core Implementation	7
6.1	Physics Engine	7
6.2	XPBD Soft Body and Cloth	7
6.3	SPH Fluid Simulation	9
6.4	Rendering Pipeline	10
6.5	Rendering Pipeline	10
6.6	Render Graph and GPU Submission	11
6.7	Engine Systems	11
6.8	Job System and Parallel Execution	12
7	Educator-Facing Visual Authoring Layer	13
7.1	Design Goals for Non-Programmer Users	13
7.2	Template System Architecture	13
7.3	Editor Interface Design	13
7.4	Editor Interface Design	13
7.5	Natural Language Shader Authoring	14
7.6	Pedagogical Scaffolding	14
7.7	Export and Classroom Deployment	15
8	Passive Reinforcement Learning Auto-Tuner	15
8.1	System Integration Overview	15
8.2	Telemetry and State Encoding	15
8.3	Policy Networks and Safety Gates	15
8.4	Session Boundary Application Model	17

9	Results	17
9.1	Prototype Demonstrations	18
9.1.1	Phase 1: Verlet Cloth Prototypes	18
9.1.2	Phase 2: Robosim Robotics Prototype	18
9.1.3	Phase 3: Matrix Transform Visualizer	18
9.2	Simulation Subsystem Results	19
9.2.1	XPBD Cloth and Soft Body	19
9.2.2	Mass-Spring Flag Simulation	20
9.2.3	GPU Particle System	20
9.2.4	SPH Fluid Demonstration Prototypes	21
9.3	Editor Authoring Layer Results	22
9.3.1	Natural Language Shader Generation	23
9.3.2	LLM-Assisted CMake Generation	24
9.4	RL Auto-Tuner Results	26
9.4.1	Parallelism Baseline	26
9.4.2	Frame Scheduler Environment	27
9.4.3	Job Scheduler Environment	27
10	Conclusion and Future Work	29
10.1	Conclusion	29
10.2	Future Work	29
10.2.1	NeRF-Based Avatar Integration	30

Chapter 1

Introduction

Physics education requires learners to connect symbolic formalisms such as differential equations, constraint systems, and conservation laws with observable behaviours, including oscillation, damping, collision response, and stability. While symbolic reasoning provides analytical structure, intuition often develops only when systems are visualised and their responses observed dynamically. Interactive demonstrations shorten the feedback loop between parameter adjustment and behavioural outcome, allowing learners to explore cause and effect in real time. This interaction is especially valuable for phenomena that are difficult to reproduce physically in classrooms, including complex multi-body collisions, large-scale fluid behaviour, and systematic parameter sweeps across stiffness, friction, and restitution. Interactive simulation, therefore, functions not merely as a visual aid but as an experimental medium for conceptual understanding.

Despite their pedagogical benefits, interactive demonstrations face significant production and deployment barriers. Classroom tools must be quick to create with limited preparation time, easy to modify without knowledge of the engine internals, reliable on older, resource-constrained machines, and aligned with instructional objectives. Adoption research consistently identifies limited technology resources, insufficient preparation time, and integration difficulty as primary obstacles to the use of digital simulations [1, 2]. Ease of use and compatibility with school hardware emerge as decisive factors in adoption decisions.

Three forms of friction commonly impede classroom deployment. Authoring friction arises when creators must understand programming languages, engine abstractions, and asset pipelines before producing a simple demonstration. Deployment friction occurs when distributing binaries, resolving driver dependencies, and handling platform differences becomes burdensome. Performance fragility occurs when a demonstration that runs smoothly on development hardware becomes unstable or stuttery on classroom machines. These constraints redirect effort from pedagogy toward engineering and discourage sustained adoption.

Prior tools have attempted to reduce programming barriers. Easy Java Simulations enabled educators to focus on mathematical models while abstracting much of the underlying engineering complexity [3]. However, classroom expectations have evolved. Educators increasingly seek real-time three-dimensional physics, modern rendering, and richer interactivity delivered reliably across heterogeneous hardware. Contemporary game engines provide these capabilities but assume developer-level expertise and modern hardware resources. AI-assisted code generation has reduced initial authoring cost by enabling rapid prototype creation from natural language descriptions [4], yet it does not resolve concerns related to packaging, deterministic behaviour, performance stability, or long-term maintainability. A gap, therefore, persists between educator accessibility and engine-level robustness.

This thesis addresses that gap through the design and implementation of LAGEngine, a unified real-time three-dimensional engine organised into three integrated layers. The engine core is a custom C++17 system supporting rigid body dynamics, XPBD based cloth and soft bodies, SPH fluids, constraint based joints, a deferred physically based rendering pipeline, an Entity Component System with JSON serialization, Lua scripting, skeletal animation, spatial audio, robotics based on Denavit Hartenberg kinematics and Jacobian inverse kinematics with proportional integral derivative control, and a multithreaded job system built on Chase Lev work stealing. Above this core sits an educator-facing visual editor implemented with ImGui, through which non-programmer users construct demonstrations using preset templates such as cloth with wind, projectile motion, SPH sandboxes, pendulum systems, inverse kinematics arms, and rigid body stacks, while modifying parameters through structured panels without touching engine code. The third layer is a passive reinforcement learning auto-tuner embedded within the runtime that observes telemetry, including frame time, GPU time, and job queue pressure, and applies conservative configuration adjustments between sessions to promote stable performance on low-power classroom hardware.

The core claim is that a physics demonstration platform can be both easy for non-experts to build with and viable on low-power hardware, provided you pair constrained template authoring with a performance-conscious runtime and

passive metrics-driven tuning. To test this, the work asks what minimal simulation primitives cover common classroom demos, which parts of a physics-heavy frame parallelise safely, which telemetry signals yield stable adaptive behaviour, and which packaging strategies reduce deployment failures on older machines. The system contributes a teacher-first authoring workflow, a multithreaded runtime with render-graph-guided GPU submission, a passive auto-tuner with reinforcement learning as an adaptive extension, and an evaluation methodology grounded in classroom-realistic constraints. The rest of the thesis walks through architecture, implementation, and results.

Chapter 2

Background and Related Work

2.0.1 Interactive Physics Education Tools

Interactive simulations have consistently demonstrated positive effects on student engagement and conceptual understanding. The PhET project showed that carefully designed interactive environments support construction of mental models more effectively than passive instruction [5]. Research across multiple scientific fields shows that students learn more when they can adjust simulation parameters and see the results immediately [6].

Authoring and deployment remain barriers. Easy Java Simulations lowered the threshold for creation by enabling educators to focus on mathematical modelling while abstracting software engineering complexity [3]. This reinforced the principle that scientific intent should dominate interface design. However, EJS was developed under earlier computational expectations and largely supports two-dimensional simulations.

Teachers often struggle to adopt simulations because they lack time, resources, and easy ways to fit them into existing courses [1, 2]. AI tools can speed up the initial creation step [4], but the output is not guaranteed to run reliably, perform well, or be easy to modify later. Getting from a rough prototype to something actually deployable is still an unsolved problem.

2.0.2 Physics Simulation Methods

Real-time physics always involves a tradeoff between accuracy, stability, and speed. Mass-spring cloth models are easy to implement but blow up at high stiffness unless you use very small timesteps. Implicit integration fixes the stability problem but requires solving large linear systems every frame [7].

Position-Based Dynamics took a different approach by treating simulation as iterative constraint projection, which made it unconditionally stable even with large timesteps [8]. The downside was that the material behaviour changed with the timestep. Extended Position-Based Dynamics fixed this by adding explicit compliance to constraints, so materials behave consistently regardless of timestep or iteration count [9].

Smoothed Particle Hydrodynamics provides a mesh-free Lagrangian approach suitable for interactive free surface flows. Müller et al. demonstrated interactive SPH for graphics [10], and Ihmsen et al. surveyed subsequent refinements [11]. Constraint-based rigid body contact formulations established by Baraff underpin modern sequential impulse solvers [12].

XPBD and SPH were selected for LAGEngine because they provide stable, real-time behaviour with tunable parameters that map cleanly onto adaptive control objectives.

2.0.3 Real Time Rendering Architectures

Deferred rendering stabilises cost by decoupling lighting from geometry through G Buffer storage and subsequent lighting passes [13]. Physically based rendering builds on microfacet theory introduced by Cook and Torrance [14] and grounded in the rendering equation [15]. GGX-based shading [16] and image-based lighting [17] enable visually consistent illumination under parameter variation.

Screen-space ambient occlusion approximates local occlusion using G-Buffer data and provides a controllable trade-off in quality [18]. Render graph architectures express frame structure as directed acyclic graphs, supporting

predictable execution and resource management [19].

OpenGL CPU overhead can dominate performance on low-end systems. Persistent mapped buffers [20], multi draw indirect submission [21], and parallel shader compilation [22] reduce driver cost [23]. Because OpenGL restricts a rendering context to one thread [24], data preparation is parallelised while submission remains centralised. Explicit APIs such as Vulkan [25], Direct3D 12 [26], and Metal [27] allow parallel command recording but increase implementation complexity. Frame pacing is evaluated using percentile metrics and tools such as PresentMon [28].

2.0.4 Game Engine Architecture

Layered engine organisation has become a practical standard because it separates concerns that would otherwise create brittle coupling between performance-critical subsystems [13]. The Entity Component System reflects a data-oriented philosophy: by grouping components of the same type in contiguous memory, iteration patterns become predictable and cache-friendly, which matters considerably on hardware where memory latency dominates compute cost. Work-stealing schedulers underpin parallel execution in most serious engines, with both strong theoretical foundations [29] and well-understood practical implementations [30].

Commercial engines have converged on similar conclusions through independent paths. Unreal dedicates separate threads to simulation and rendering, recognising that coupling the two creates unpredictable frame times [31,32]. Unity exposes job dependencies explicitly to the programmer, making parallelism a first-class authoring concern rather than an implementation detail [33,34]. Godot takes a more conservative approach, documenting precisely which APIs are thread-safe and requiring the developer to reason carefully about what can run concurrently [35]. The common thread across all three is that CPU-side work is aggressively distributed across cores, while GPU command submission is kept centralised and disciplined to avoid driver contention.

2.0.5 Reinforcement Learning for System Optimisation

Reinforcement learning has been applied to configuration tuning in systems with hardware-dependent performance characteristics. OtterTune demonstrated learned configuration recommendation for databases [36]. CDBTune and Athena extended this approach using deep reinforcement learning methods [37,38]. These works show that learned controllers can outperform static defaults in nonlinear parameter spaces.

Engine parameters such as solver iterations and particle caps interact with workload and hardware in similarly nonlinear ways. Proximal Policy Optimisation provides stable updates [39], while Deep Deterministic Policy Gradient supports continuous control [40]. Constrained Policy Optimisation and safe reinforcement learning research emphasise the explicit use of safety boundaries in real deployments [41,42]. Reproducibility concerns require multi seed evaluation and variance reporting [43].

2.0.6 Synthesis and Identified Gap

Educational research supports interactive simulations but highlights accessibility and hardware constraints as determinants of adoption [1,2,5,6]. Simulation research provides stable methods such as XPBD and SPH [8–10]. Rendering and engine architecture research provide disciplined performance-oriented structures [13,14,19]. Systems research shows that learned configuration can outperform static heuristics when applied carefully [36,39].

These strands remain largely disconnected in practice. Educator tools lack modern physics depth and adaptive performance, commercial engines assume developer expertise, and reinforcement learning tuners are not embedded within educator-facing simulation systems. No existing platform simultaneously integrates accessible three-dimensional authoring, a custom XPBD and SPH physics core, and a passive reinforcement learning auto-tuner for heterogeneous classroom hardware. LAGEngine is designed to satisfy these combined constraints through a unified architecture linking template-driven authoring, performance-oriented runtime design, and adaptive configuration selection.

Chapter 3

Problem Statement and Design Goals

LAGEngine addresses the lack of a system that allows educators to create interactive three-dimensional physics demonstrations without engine-level programming expertise while maintaining stable performance on heterogeneous classroom hardware. Existing engines assume developer-grade skills and complex asset pipelines, whereas lightweight educational tools lack modern physics depth and performance scalability. Artificial intelligence-assisted authoring reduces prototype creation cost [4] but does not resolve packaging, determinism, or hardware robustness. Classroom machines are frequently older and constrained, and demonstrations that perform well on development hardware may fail under real deployment conditions. Although reinforcement learning has been applied to configuration tuning in complex systems [38], it has not been integrated into educator-facing physics platforms. This work claims that accessibility and hardware-constrained performance can be addressed simultaneously through template-driven authoring, a multithreaded AZDO (Approaching Zero Driver Overhead)informed runtime, and a passive reinforcement learning auto-tuner that applies bounded adjustments between sessions.

The system must enable non programmer educators to generate, modify, and export demonstrations within minutes; execute reliably on Windows, Linux, and macOS machines including those with integrated graphics and limited resources; maintain deterministic and reproducible simulation behavior without mid session adaptation; and stabilize frame time to at least thirty frames per second on low end hardware and sixty frames per second on mid range systems. LAGEngine does not aim to compete with commercial engines in feature breadth, support networking or mobile deployment, provide film quality rendering, or perform online reinforcement learning during live sessions. This trade-off favours stability and accessibility over maximal extensibility.

Chapter 4

Prototype Evolution and Design Rationale

4.0.1 Phase 1: Verlet Cloth Prototypes

The first prototypes implemented cloth as a mass-spring grid, integrated with Verlet updates. A two-dimensional version simulated gravity and damping with interactive cannon impacts, while a three-dimensional extension introduced classes such as `Cloth3D`, `Particle3D`, and `Ball3D` and used iterative position projection for constraint correction. Rendering and physics were tightly coupled, and the stiffness parameters were time-step-dependent and nonphysical.

Large timesteps led to instability, and increasing stiffness only amplified divergence rather than enhancing realism. Iterative projection improved stability and reflected ideas later formalised in Position Based Dynamics [8], yet the absence of explicit compliance parameters limited control. These failures motivated the adoption of Extended Position Based Dynamics [9], whose timestep invariant formulation resolved the calibration and stability issues exposed by the prototype.

4.0.2 Phase 2: Robosim Robotics Prototype

The second prototype implemented a serial robot arm using Denavit Hartenberg parameters [44] to describe link geometry and forward kinematics. Inverse kinematics employed a Jacobian transpose solver, and a PID controller ensured smooth actuation. All geometry was generated procedurally through simple mesh classes rather than external assets.

This work demonstrated that structured kinematic data maps cleanly onto a component-based model and that Jacobian transpose methods are sufficient for interactive demonstrations. Procedural geometry proved visually adequate without an asset pipeline, supporting an educator-focused workflow. Architectural lessons aligned with established engine design principles [13] and directly informed the Robotics module of the final engine.

4.0.3 Phase 3: Bullet Physics Evaluation and Rejection

A structured exploration of Bullet Physics produced seventeen demo applications covering rigid bodies, constraints, soft bodies, and advanced features [45]. Although functionally rich, the API required substantial boilerplate and managed its own object lifecycle, which complicated integration with a UUID-based ECS.

The solver operated as a black box, limiting insight and tunability at the level required for adaptive auto-tuning. These constraints led to the rejection of Bullet in favour of a custom physics core. Bullet demos were retained as behavioural references, but the final system implements physics internally.

4.0.4 Design Principles Derived from Prototyping

The prototypes established four governing principles. Stability takes precedence over feature breadth because unstable simulation undermines educational value. An ECS-first architecture is essential to avoid tight coupling and ensure coherent ownership across subsystems. Solver transparency and tunability are necessary so that XPBD compliance and SPH particle counts can be exposed to the auto tuner. Procedural asset generation removes pipeline friction and keeps the engine accessible to educators. These principles define the architectural foundation of LAGEngine.

Chapter 5

System Architecture Overview

5.1 Architecture Overview

LAGEngine is organized into six layers, shown in Figure 5.1. Each layer has a clear responsibility and communicates with adjacent layers through well-defined interfaces.

Layer 1: Educator Authoring Layer. This is the top-level interface that teachers interact with. It provides a template library of preconfigured demos, a scene editor built on Dear ImGui, parameter controls for tuning simulation values, simulation presets for common setups, and an export and packaging system that serializes scenes into standalone JSON files. The editor operates directly on the same ECS scene representation used at runtime, with no intermediate abstraction in between.

Layer 2: Application Layer (Simulation Content). This layer holds the actual demonstrations built using the authoring tools: physics demos, classroom experiments, robotics demos, fluid simulations, and collision experiments. These are compositions of entities and components defined in the layers below.

Layer 3: Entity Component System (ECS). All simulation state lives here as typed component pools indexed by UUID-stable entities. The core components include Transform, RigidBody, Collider, MeshRenderer, Camera, Light, ScriptComponent, and AudioComponent. Every system in the runtime reads from and writes to these components.

Layer 4: LAGEngine Runtime Core. This is the main execution layer, split into six subsystems. The physics engine handles collision, constraints, XPBD cloth, and SPH fluids. The rendering engine runs a deferred PBR pipeline with shadow mapping and post-processing. The multithreading subsystem provides a job scheduler with work-stealing. Scripting is handled through a Lua runtime. The robotics and animation subsystem covers skeletal animation, DH kinematics, Jacobian IK, and PID controllers. Audio runs as a separate subsystem alongside these.

Layer 5: Adaptive RL Auto-Tuner. This layer sits below the runtime and observes it passively. It collects telemetry, performs frame analysis and load monitoring, runs an RL policy over aggregated metrics, and writes configuration adjustments that take effect at the next startup. It does not modify parameters during live execution, which preserves deterministic behaviour.

Layer 6: Platform and External Libraries. The bottom layer handles platform abstraction. It includes OpenGL for graphics API access, GLFW for windowing, Dear ImGui for UI, math libraries, asset loading, and

filesystem access.

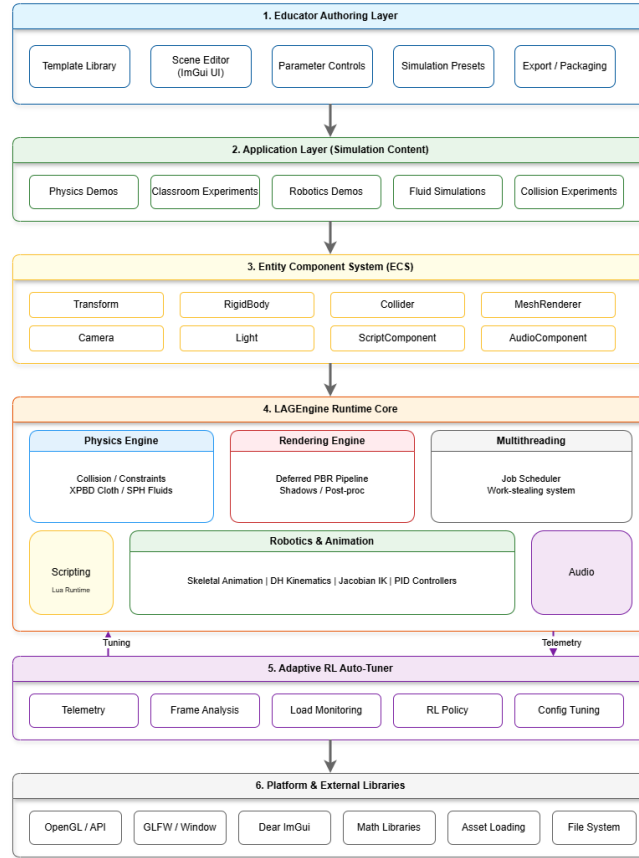


Figure 5.1: LAGEngine architecture showing all six layers from the educator authoring interface down to platform and external library dependencies.

5.2 Data Flow Across Layers

Each frame, simulation systems in Layer 4 read and write ECS components in Layer 3 (transforms, rigid bodies, XPBD constraints, SPH particles, robotics state). The rendering engine then reads the updated transforms, mesh data, and material parameters to build indirect draw buffers and submit GPU commands through the render graph.

Telemetry flows downward from Layer 4 to Layer 5. CPU frame depth comes from high-resolution timers, GPU duration from timer queries, and job system statistics include queue depth, steal counts, and worker utilisation. These metrics are written into a ring buffer that retains recent history.

Between sessions, the RL module in Layer 5 reads aggregated telemetry through a pybind11 bridge, builds a state vector from exponentially weighted moving averages, and runs policy inference to produce candidate configurations (solver iterations, resolution scale, shadow quality, batching thresholds). Safety gates validate the output before writing a config file that Layer 4 consumes on the next startup.

Tuning flows upward from Layer 5 back into Layer 4, while telemetry flows downward from Layer 4 into Layer 5, forming a closed feedback loop across sessions.

5.3 Deployment Model

The engine core is compiled as a static C++17 library. The editor and example applications link directly against this library, ensuring identical runtime behaviour between development and exported builds. Scene state is serialised

into JSON using UUID-stable references, and assets are packaged into a self-contained archive.

Export produces a standalone executable bundle containing the engine binary, scene description, shaders, and required assets. No external installation steps are required beyond execution of the binary. The platform abstraction layer supports Windows, Linux, and macOS using OpenGL 4.x, enabling compatibility with integrated GPUs and older classroom hardware.

Chapter 6

Engine Core Implementation

6.1 Physics Engine

The rigid body subsystem handles the motion and rotation of objects. Each frame, forces and torques are accumulated in each body's component data from the subsystems that apply them (gravity, springs, user input, etc.). Once all forces are collected, a semi-implicit Euler integrator updates linear and angular velocities first, then advances position and orientation. Inertia tensors are stored in body-local space and rotated into world space at every frame, so that rotational responses remain physically correct as objects tumble. The key design choice here is that force accumulation and integration are separate steps. Any subsystem can write forces into component data without needing to know how integration works, keeping the code decoupled.

Collision detection runs in two stages. The broad phase uses a spatial hash that divides world space into fixed-size grid cells. Each object is inserted into whichever cells it overlaps. To find potential collisions, the system only checks objects that share a cell, reducing the cost from a naive $O(n^2)$ pairwise check to roughly $O(n)$ in practice. The cell size is chosen to match the typical object diameter so most objects land in one or two cells.

Pairs that survive the broad phase move into narrow-phase testing. The engine supports six shape-pair combinations: box versus box, sphere versus sphere, box versus sphere, capsule versus capsule, and plane interactions with the other shapes. Each narrow-phase test produces a contact manifold, a small data structure that stores the penetration depth (how far the objects overlap), the contact normal (the direction to push them apart), and one or more contact points on the surface.

Contact manifolds are then fed into a sequential-impulse solver based on the Gauss-Seidel method, consistent with constraint-based formulations [12]. The solver iterates over all contacts multiple times per frame. On each pass, it computes impulses that push overlapping objects apart along the contact normal (non-penetration), resist sliding along the contact surface (friction), and applies both velocity-level and positional corrections. The positional correction step is important because velocity-only solving allows small amounts of drift to accumulate over time, eventually causing objects to visibly sink into each other.

Joint constraints (distance, fixed, hinge, and slider) are solved in the same iterative loop alongside contacts. Distance joints keep two anchor points at a fixed separation. Fixed joints lock relative position and orientation entirely. Hinge joints allow rotation around a single axis. Slider joints allow translation along a single axis. Treating joints and contacts uniformly in one solver pass means they naturally interact with each other without special-case code.

6.2 XPBD Soft Body and Cloth

Extended Position-Based Dynamics (XPBD) improves on standard PBD by adding an explicit compliance parameter to each constraint [8, 9]. Compliance controls how stiff or soft a constraint is. The important property is that stiffness stays consistent regardless of the timestep or how many solver iterations you run. In standard PBD, cutting the iteration count makes materials feel softer, which is a problem. In XPBD, cutting iterations degrades quality gracefully but the material still behaves like the same material. This is what makes XPBD a good fit for the auto-tuner: when the system detects that hardware is struggling, it can reduce solver iterations to save time without

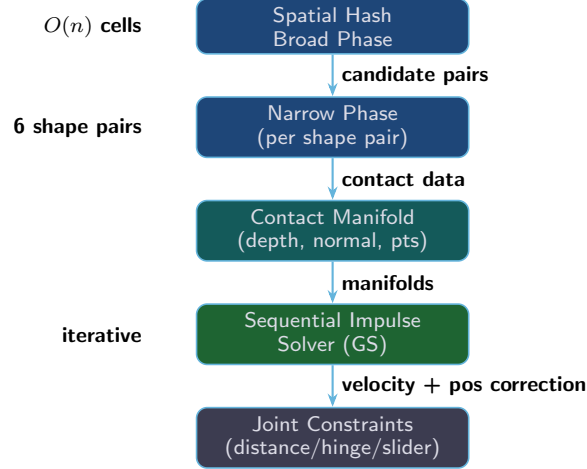


Figure 6.1: Collision detection and resolution pipeline.

changing how the cloth or soft body feels to the user.

The solver loop works as follows. First, every particle’s position is predicted forward using its current velocity: $\tilde{x} = x + \Delta t v$. These predicted positions are then iteratively corrected by projecting three types of constraints. Distance constraints keep connected particles at their rest-length separation, preventing cloth from stretching unrealistically. Bending constraints resist folding between adjacent triangles, controlling how stiff or floppy the material looks. Volume constraints maintain the overall volume of soft bodies to prevent collapse under pressure.

Each constraint projection computes a Lagrange multiplier update $\Delta\lambda$, which determines how far each affected particle should move. This update is scaled by the constraint’s compliance value α and the timestep Δt . Low compliance means stiff behaviour (the constraint is enforced tightly), high compliance means soft behaviour (the constraint allows more stretch). The projection and multiplier update steps repeat for a configurable number of iterations. More iterations produce more accurate results but cost more time.

After all iterations finish, velocities are reconstructed from the difference between the corrected positions and the original positions: $v \leftarrow (\tilde{x} - x) / \Delta t$. Deriving velocities from position changes rather than tracking them independently ensures that momentum stays consistent with where particles actually ended up.

Two additional features sit atop this loop. Cloth tearing checks each distance constraint after solving and deletes any constraint whose stretch ratio exceeds a configurable threshold τ . Once deleted, the particles on either side are no longer connected, which creates a visible rip in the mesh. Self-collision is handled by an `OctreeAccelerator` that spatially partitions particles into a tree structure. Instead of testing every particle against every other particle, the narrow-phase self-collision tests only run between particles that occupy nearby tree nodes, keeping the cost manageable as particle counts grow.

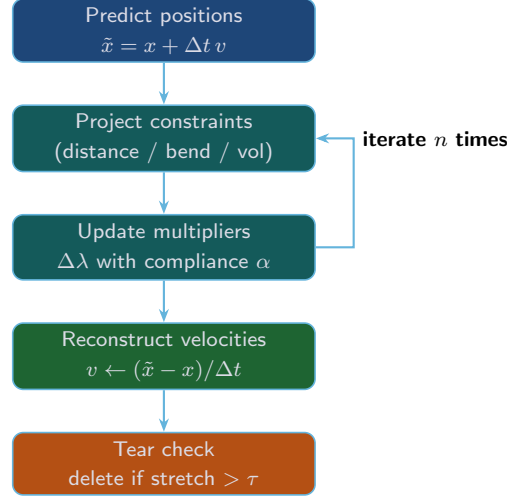


Figure 6.2: XPBD constraint projection loop.

6.3 SPH Fluid Simulation

Fluid simulation uses Smoothed Particle Hydrodynamics (SPH), a meshfree method where the fluid is represented as a collection of particles rather than a fixed grid [10, 11]. Each particle carries mass, position, velocity, and density. The behaviour of the fluid emerges from how these particles interact with their neighbours.

The simulation runs in four stages each frame. The first stage finds neighbours. Each particle only interacts with other particles within a small radius h called the smoothing length. To avoid checking every particle against every other particle, the system reuses the same spatial hash grid used by the rigid body collision system. Particles are inserted into grid cells based on their position, and neighbour queries only look at adjacent cells. This also means the job system can split density and force calculations across worker threads in parallel, since the spatial hash is already shared infrastructure.

The second stage estimates density. Each particle's density ρ_i is computed by summing weighted contributions from all neighbours using the poly6 kernel function. Particles in crowded regions end up with high density, particles in sparse regions end up with low density. Getting density right is critical because every other force depends on it.

The third stage computes forces. Pressure forces push particles apart when local density exceeds the rest density ρ_0 . Pressure is computed from a simple equation of state:

$$p_i = k (\rho_i - \rho_0),$$

where k is a stiffness constant that controls how strongly the fluid resists compression. The gradient of the pressure field is evaluated using the spiky kernel, which has a sharp peak at close range and produces the repulsive forces that keep particles from collapsing into each other. Viscosity forces resist relative motion between neighbouring particles, making the fluid flow smoothly rather than behaving like a collection of independent particles. These are computed using a Laplacian kernel. Surface tension is handled through additional force terms that pull surface particles inward, and boundary conditions are enforced by applying response forces when particles collide with static geometry.

The fourth stage integrates. All accumulated forces are applied using the same semi-implicit Euler integrator used by the rigid body system. Velocities are updated first from forces, then positions are advanced from the updated velocities.

For rendering, the engine does not try to build a triangle mesh from the particles. Instead, a screen-space fluid renderer takes the particle positions, renders them as depth values, smooths the depth buffer to remove individual particle shapes, reconstructs surface normals from the smoothed depth, and then shades the result through the deferred physically based rendering pipeline. This produces a continuous fluid surface without the cost of explicit

mesh reconstruction.

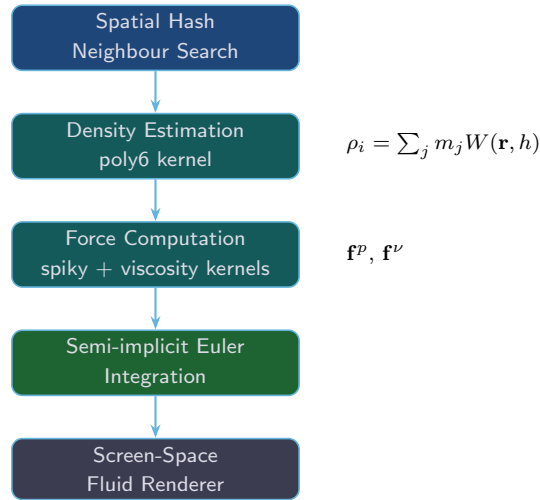


Figure 6.3: SPH simulation pipeline.

6.4 Rendering Pipeline

6.5 Rendering Pipeline

The engine uses a deferred rendering pipeline, which means geometry and lighting are handled in separate passes rather than all at once. This separation is important because it makes the cost of lighting independent of how much geometry is in the scene, which keeps frame times stable even when the physics simulation is pushing a lot of objects.

The first pass is the geometry pass. Every visible object is drawn once, and instead of computing final pixel colours, the pipeline writes surface information into a set of textures called the G-Buffer. These textures store world-space position (RGB32F), surface normals (RGB16F), albedo colour and roughness (RGBA8), metallic and ambient occlusion values (RG8), and depth (D24S8). After this pass, the GPU has a complete description of every visible surface without having processed any lights yet.

The lighting pass reads from the G-Buffer and computes final shading for each pixel. The shading model is Cook-Torrance, a physically based BRDF that combines three terms [14–16]. The GGX normal distribution function controls how tight or spread out specular highlights are based on surface roughness. Smith-GGX geometric shadowing accounts for microfacet self-occlusion at glancing angles. Fresnel-Schlick reflectance controls how much light reflects versus enters the surface depending on viewing angle. Together these three terms produce materials that respond to light in a physically plausible way.

For indirect lighting, the engine uses image-based lighting with preintegrated environment maps [17]. An environment map is pre-convolved at multiple roughness levels so that looking up indirect specular reflections at runtime is a single texture sample rather than an expensive integration.

Shadows come from two sources. Directional lights (like sunlight) use cascaded shadow maps, which split the view frustum into distance-based slices with progressively lower resolution farther from the camera. Point lights use omnidirectional cube-map shadows, rendering the scene into six faces of a cube map from the light’s position.

Screen-space ambient occlusion adds contact shadows in corners and crevices where ambient light would be partially blocked [18]. It works by sampling a hemisphere of points around each pixel in view space, testing how many of those samples fall inside nearby geometry, and then applying a separable blur pass to smooth the result.

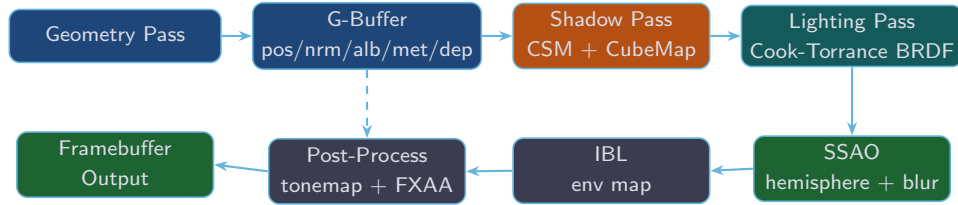


Figure 6.4: Deferred rendering pipeline pass sequence.

6.6 Render Graph and GPU Submission

The render graph organises each frame as a directed acyclic graph of render pass nodes [19]. Each node declares what render targets it reads from and what render targets it writes to. The graph resolves dependencies automatically and executes passes in topological order. This means nobody has to manually manage framebuffer binding order or figure out which pass needs to run before which. It also enables the system to reuse GPU memory across passes that do not overlap, since the graph knows exactly when each resource is live.

GPU submission is designed around the Approaching Zero Driver Overhead (AZDO) philosophy [23], which minimises the per-frame cost of talking to the graphics driver.

The first technique is persistent mapped buffers. Instead of mapping and unmapping GPU buffers every frame to upload instance data (transforms, material indices, etc.), the engine allocates ring buffers once using `GL_ARB_buffer_storage` and keeps them mapped for the lifetime of the application [20]. Each frame writes to the next section of the ring, avoiding the overhead of repeated map and unmap calls.

The second technique is indirect drawing. Rather than issuing one draw call per object, the engine packs draw parameters (vertex count, instance count, buffer offsets) into an indirect command buffer and issues a single `MultiDrawIndirect` call that processes all of them at once [21]. This amortises the driver validation cost that normally happens per draw call.

The third technique is asynchronous shader compilation. Shaders are compiled in the background using `GL_ARB_parallel_shader_compile`, which eliminates the stalls that normally happen the first time a new shader is used.

Threading follows OpenGL’s context ownership rules. OpenGL contexts can only be current on one thread at a time (`wglMakeCurrent` exclusivity [24]), so a dedicated submission thread holds the context and issues all GPU commands. Worker threads run in parallel preparing indirect command buffers, building instance data streams, and performing culling. Once the data is ready, the submission thread picks it up and sends it to the GPU.

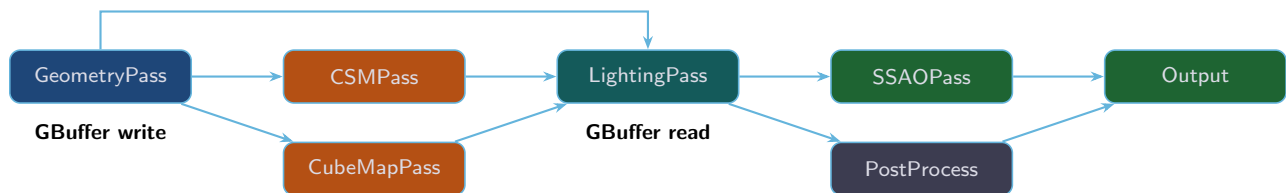


Figure 6.5: Render graph directed acyclic graph showing pass dependencies.

6.7 Engine Systems

The Entity Component System assigns every entity a 64-bit UUID and stores component data in typed pools. The system supports over twenty component types covering physics, rendering, scripting, audio, animation, and robotics. UUIDs remain stable across serialisation, so references between entities survive save and load cycles. Scenes serialise to JSON using `nlohmann/json`, and the prefab system supports deep-copy instantiation so that duplicating a complex entity hierarchy copies all of its components and child references correctly.

Scripting uses Lua 5.4 and exposes three callbacks: **OnStart** runs once when an entity is created, **OnUpdate** runs every frame, and **OnCollision** fires when the physics system detects a contact involving that entity. A file watcher monitors script files on disk and hot-reloads them when they change, so teachers can edit behaviour without restarting the engine.

Asset management is split across three systems. `ResourceManager` handles loading and caching of raw assets (textures, meshes, shaders). `AssetDatabase` tracks metadata and provides lookup by name or UUID. `PresetManager` stores reusable configuration bundles that can be applied to entities or components.

Skeletal animation interpolates between keyframes using SLERP for rotations and linear interpolation for positions. Skinning matrices are computed and uploaded to the GPU every frame so that mesh vertices deform to follow the skeleton.

Spatial audio uses OpenAL and reads world positions directly from each entity's `TransformComponent` every frame. As objects move in the simulation, their sound sources move with them automatically.

The robotics subsystem models articulated arms using Denavit-Hartenberg parameterisation [44], which defines each joint as a sequence of rotations and translations along standardised axes. Inverse kinematics uses the Jacobian-transpose method [46] to compute joint angles that move the end effector toward a target position. Joint motors are driven by discrete PID controllers stored in `RobotArmComponent`, which compute corrective torques from the error between desired and actual joint angles each frame.

6.8 Job System and Parallel Execution

The job system distributes work across all available CPU cores using a work-stealing architecture [29, 30]. Each worker thread owns a private double-ended queue (deque). When a thread finishes its own work, it steals tasks from the back of another thread's deque. This keeps all cores busy without requiring a central task queue that would become a contention bottleneck.

The main thread handles three responsibilities that cannot be parallelised: processing input events, updating the Dear ImGui interface, and submitting GPU commands (since the OpenGL context is bound to this thread). Everything else runs on workers.

Worker tasks include physics integration and constraint solving, XPBD soft-body projection, SPH fluid computation, inverse kinematics for robotic arms, transform hierarchy propagation, broad-phase spatial hash updates, frustum culling, instance data stream construction, and indirect draw buffer population. These tasks are structured as a dependency graph: physics must finish before IK runs, IK must finish before culling, and culling must finish before indirect buffers are built. Once the buffers are ready, the main thread picks them up and submits them to the GPU.

This decomposition follows established multi-threaded engine patterns [31] and respects the constraint that only one thread can own the OpenGL context at a time, while still keeping worker threads saturated with useful computation every frame.

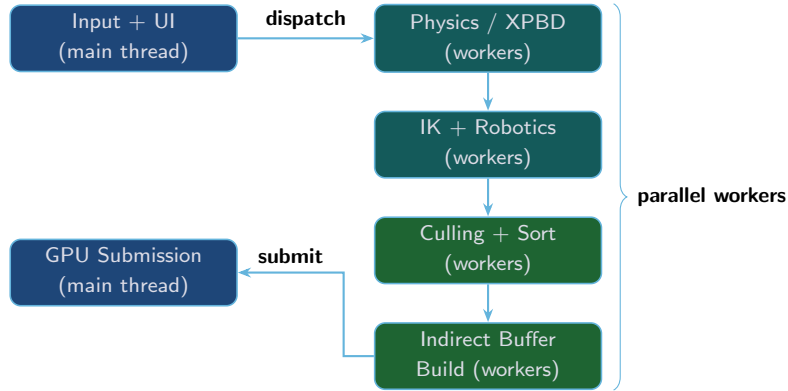


Figure 6.6: Per-frame job graph showing main-thread responsibilities and paralised worker tasks.

Chapter 7

Educator-Facing Visual Authoring Layer

7.1 Design Goals for Non-Programmer Users

The people this tool is built for mainly are teachers who know their subject but have never touched engine code. Research on why educators avoid simulations in the classroom consistently points to the same problems: it takes too long to prepare, the technical overhead is too high, and fitting new tools into an existing course is painful [1, 2]. So the editor has three hard requirements: getting a working demo should take minutes not hours, no code should ever be visible to the user, and no engine concept (entities, components, render passes) should leak through the interface. This design extends the accessible authoring model established by tools such as Easy Java Simulations [3] to real-time 3D physics.

7.2 Template System Architecture

PresetManager lets teachers launch full physics demonstrations with a single click. The available templates cover cloth simulation with wind, projectile motion, SPH fluid sandbox, pendulum constraints, robot arm inverse kinematics, and rigid body stacking. Each template spawns a complete entity hierarchy with sensible defaults already set. Every parameter is exposed through structured inspector panels, so a teacher can tweak values like gravity, stiffness, or wind strength without ever opening a code file.

7.3 Editor Interface Design

7.4 Editor Interface Design

The editor uses a docked Dear ImGui layout that follows the conventions of major game engines like Unity and Unreal [47]. Scene hierarchy sits on the left, the 3D viewport fills the center, and the component inspector occupies the right. This layout is deliberately familiar so that anyone who has used a modern game engine (or watched a tutorial for one) can orient themselves immediately.

SceneHierarchyPanel displays the entity tree and supports drag-to-reparent and multi-selection. ViewportPanel renders the live 3D scene with mouse-driven navigation and a TransformGizmo for moving, rotating, and scaling objects by ray-cast selection. ComponentsWindow displays typed fields for the selected component. AssetBrowserPanel manages resources like meshes, textures, and materials. ConsolePanel logs runtime events. ProfilerWindow displays

frame and subsystem timing so teachers (or developers) can see where time is being spent. GraphicsSettingsPanel exposes quality knobs like shadow resolution, SSAO, and particle limits. ToolbarPanel provides play, pause, stop, and frame-step controls for running and debugging simulations.

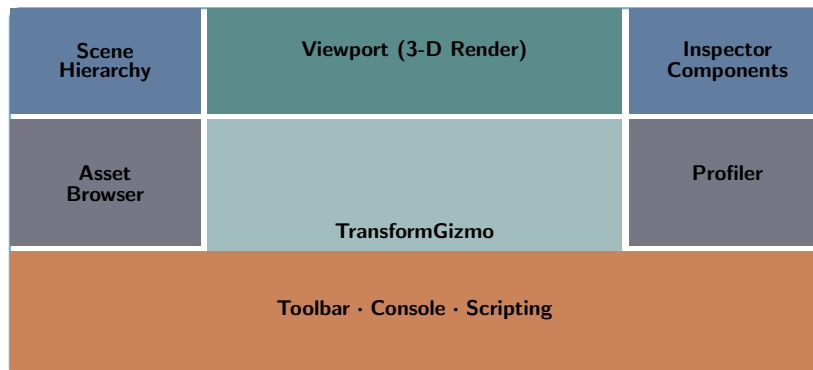


Figure 7.1: Schematic layout of the Dear ImGui editor interface.

7.5 Natural Language Shader Authoring

The editor includes an optional workflow for creating GLSL shaders from plain-language descriptions, so teachers can define custom surface appearances without writing GPU code. A material panel in the editor lets the user type something like “animated fire effect” or “toon shading with four bands.” This description is sent to a local large language model running through Ollama [48], using the Qwen2.5-Coder 7B model [49]. Running inference locally means the feature runs offline and does not depend on external APIs or cloud costs.

The prompt sent to the model enforces GLSL 330 core compatibility and a fixed vertex-fragment interface. Examples retrieved from a curated shader library are included in the prompt to guide the model toward the correct structure and style. The generated shader pair is compiled immediately in a live OpenGL context. If compilation fails, an automatic repair loop retries before exposing the result to the user, so only validated programs are ever applied to scene entities. The resulting material can be previewed in the viewport, edited by hand if needed, and exported alongside the scene.

This same local LLM infrastructure is also used for CMake build file generation and for general code assistance during development. AI-assisted coding tools like Cursor [50] and Claude Code [51] have demonstrated that LLM-driven code generation can meaningfully accelerate development workflows when integrated directly into the editing environment. The shader authoring workflow extends the template-driven philosophy of the editor into rendering: instead of requiring the user to understand GLSL, it lets them describe what they want and receive inspectable, deterministic shader code that lives within the engine’s controlled runtime.

7.6 Pedagogical Scaffolding

Templates include labelled parameter fields and inline observation prompts that direct student attention toward measurable outputs (for example, “observe how increasing spring stiffness affects oscillation frequency”). A student display mode strips all editor UI and presents only the simulation view, removing distractions. Controlled experiment mode restricts each interaction to a single parameter change at a time, reinforcing the cause-and-effect relationship between a variable and what the student can see happening on screen.

7.7 Export and Classroom Deployment

Export packages the engine binary and the scene JSON into a self-contained archive that requires no additional installation on classroom machines. `WelcomePanel` and `NewProjectDialog` manage project lifecycle (creating, opening, and switching projects). Playback controls support play, pause, and single-frame stepping. A lightweight recording feature captures demonstration sequences so teachers can reuse them asynchronously, for example in a flipped classroom setup where students watch recorded demos before a lab session.

Chapter 8

Passive Reinforcement Learning Auto-Tuner

8.1 System Integration Overview

The reinforcement learning subsystem does not control the engine while it is running. It is a passive configuration advisor. The engine generates telemetry every frame and writes it into a metrics buffer that persists between sessions. After the engine shuts down, a Python-based training environment reads that telemetry through a `pybind11` bridge, runs policy inference, and writes an updated configuration file. The engine picks up this file on the next startup. At no point does the RL module touch engine parameters during a live session.

Figure 8.1 shows the full pipeline. Telemetry flows down from the engine runtime through the `pybind11` bridge into the RL module. Inside the RL module, a `StateEncoder` aggregates raw metrics into a compact state vector. A PPO policy network evaluates this state and produces an action vector of candidate configuration changes. These actions pass through a chain of five safety gates before a `ConfigWriter` commits the validated result to `config.json`, which the engine reads on its next launch.

Two Gymnasium environments, `WorkStealSchedulerEnv` and `FrameSchedulerEnv`, provide simulated workload variation for training the policy offline. These environments also feed into the state vector pipeline so that training and deployment share the same encoding path.

8.2 Telemetry and State Encoding

Telemetry collection runs inside the engine’s frame loop across six subsystems shown in the top row of Figure 8.1. `PhysicsSystem` reports solver iteration counts and active particle counts. `TimingSystem` measures CPU frame duration and GPU time using `GL_ARB_timer_query`. `JobSystem` records queue depth and steal statistics from the work-stealing dequeues. `StutterDetector` flags any frame that exceeds 33.3 ms (a 30 fps miss) or 50 ms (a 20 fps miss). `RenderSystem` logs draw call counts and GPU submission timing. `TelemetryBuffer` stores all of these signals in a ring buffer that survives across frames and is persisted to disk at session end.

Between sessions, the `pybind11` bridge reads this persisted buffer and hands it to the `StateEncoder`. The `StateEncoder` applies exponentially weighted moving averages to smooth out per-frame noise and capture recent performance trends rather than instantaneous spikes. The output is a `StateVector` with six fields: mean frame time, p95 frame time, GPU time estimate, job queue depth, steal rate, and hitch count. This is deliberately compact. A small state vector keeps policy inference fast and avoids overfitting to transient frame-level noise.

8.3 Policy Networks and Safety Gates

The policy network uses Proximal Policy Optimization (PPO) with a clipped surrogate objective [39]. The clipping is important here because it prevents the policy from making large destructive parameter changes in a single update, which matters when those parameters directly affect whether a classroom demo runs smoothly. The network itself is a straightforward MLP. Deep Deterministic Policy Gradient (DDPG) is also supported as an alternative for strictly

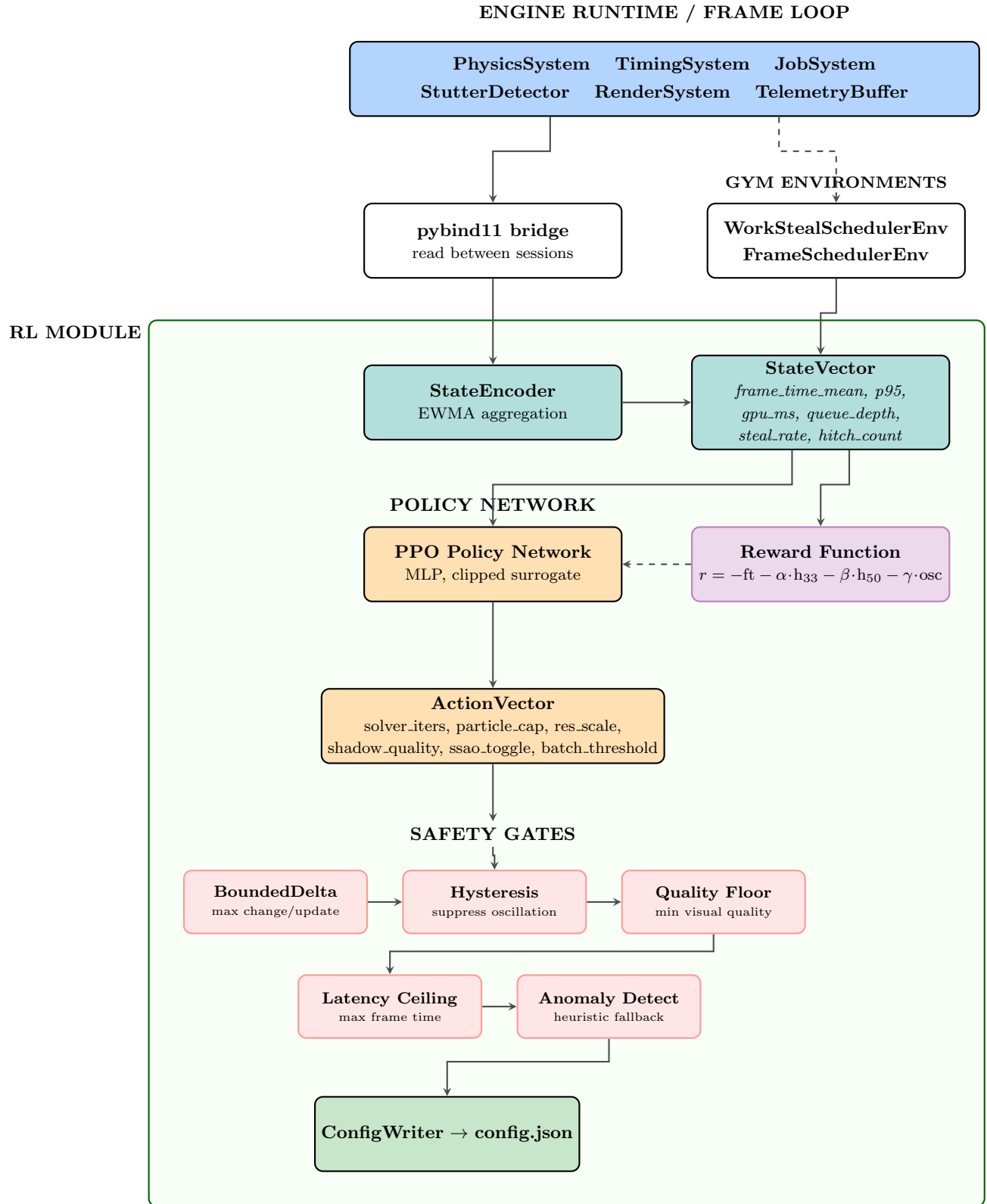


Figure 8.1: RL auto-tuner architecture: per-frame telemetry flows through pybind11 into gym environments, is encoded via EWMA, evaluated by PPO, validated through safety gates, and written to config for the next session.

continuous action spaces [40]. Training uses the two Gymnasium environments (FrameSchedulerEnv and WorkStealSchedulerEnv) to simulate workload variability, and metrics are logged through TensorBoard. Reproducibility practices follow guidance from Henderson et al. [43].

The reward function shown in Figure 8.1 is:

$$r = -\text{ft} - \alpha \cdot h_{33} - \beta \cdot h_{50} - \gamma \cdot \text{osc}$$

where ft is mean frame time, h_{33} counts frames exceeding the 33 ms threshold, h_{50} counts frames exceeding 50 ms, and osc penalises oscillation in configuration values between sessions. The weighting coefficients α , β , γ are tuned to prioritise stability over raw throughput. A configuration that holds a steady 30 fps is preferred over one that averages 60 fps but hitches regularly.

The policy outputs an action vector with six parameters: solver iteration count, particle cap, resolution scale, shadow quality level, SSAO toggle, and batching threshold. These are the knobs the auto-tuner is allowed to adjust.

Before any of these values reach disk, they pass through five safety gates arranged in sequence (shown in the bottom half of Figure 8.1):

BoundedDelta limits how much any single parameter can change per update. Even if the policy wants to cut solver iterations from 20 to 2, BoundedDelta will only allow a step of a few iterations at a time.

Hysteresis suppresses oscillation. If the policy keeps flipping a parameter back and forth between sessions, hysteresis locks the value until a consistent trend emerges.

Quality Floor enforces minimum visual quality. No matter what the policy suggests, certain settings (like minimum shadow resolution or minimum particle count) cannot drop below a defined threshold.

Latency Ceiling enforces a maximum acceptable frame time. If a proposed configuration would push frame time above the ceiling based on recent telemetry, the change is rejected.

Anomaly Detect catches cases where telemetry looks abnormal (for example, a sudden spike caused by a background OS process rather than an actual workload problem). When triggered, it falls back to a deterministic heuristic configuration instead of trusting the policy output.

Only configurations that survive all five gates are written to `config.json` by the ConfigWriter [41, 42].

8.4 Session Boundary Application Model

The separation between the RL module and the running engine is strict and intentional. During a live session, the engine reads its configuration once at startup and never modifies it. Telemetry is collected but nothing changes. The simulation behaves identically every time it runs with the same config file, which means a teacher can demonstrate the same scenario repeatedly and get the same results.

After the session ends, the RL module wakes up, reads aggregated telemetry, runs policy inference, validates the output through the safety gates, and writes an updated config file if it finds an improvement. On the next startup, the engine loads this new configuration.

This design means the auto-tuner progressively adapts to whatever hardware the engine is running on, session by session, without ever surprising the user during a live demonstration. A teacher running the engine on a ten-year-old classroom laptop will see the system gradually converge toward settings that hold a stable frame rate on that machine. A teacher on a newer workstation will converge toward higher quality settings. In both cases, the behaviour during any given session is fully deterministic and predictable.

Chapter 9

Results

This chapter presents empirical results across four evaluation domains: prototype evolution, the visual authoring editor, simulation subsystems, and the passive reinforcement learning auto-tuner. All benchmarks were conducted

on a machine running Ubuntu 22.04 with an NVIDIA GeForce GTX 1650 and 11 logical CPU cores. Frame time is reported in milliseconds; lower values are better. Standard deviation and p95 latency are included where relevant to characterise stability rather than peak performance alone.

9.1 Prototype Demonstrations

9.1.1 Phase 1: Verlet Cloth Prototypes

The first prototypes implemented mass-spring cloth using Verlet integration. Figure 9.1a shows the undisturbed cloth mesh under gravity, demonstrating stable draping. Figures 9.1b and 9.1c show progressive constraint deletion after successive cannon impacts, exposing the tearing behaviour that motivated the transition to XPBD. At 59 FPS the simulator ran comfortably in real time, but stiffness calibration was timestep-dependent and high spring constants produced visible divergence, directly motivating the adoption of Extended Position-Based Dynamics.

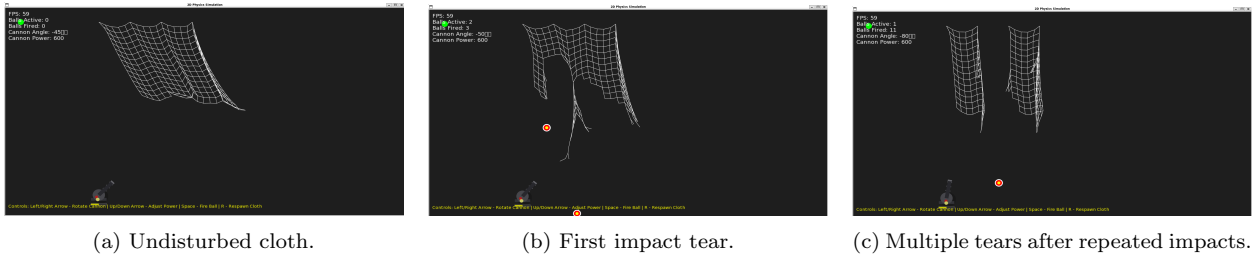


Figure 9.1: Phase 1 Verlet cloth prototype showing tearing under cannon impact.

9.1.2 Phase 2: Robosim Robotics Prototype

The robotics prototype implemented a six-joint serial arm using Denavit–Hartenberg parameters and Jacobian-transpose inverse kinematics. Figures 9.2a and 9.2b show the arm at two configurations with the TCP position, joint PID gains, and DH parameter table visible. The prototype confirmed that Jacobian-transpose methods are sufficient for interactive demonstrations and that procedural geometry removes the need for an external asset pipeline. The performance overlay reported 152 ms per frame (6.6 FPS), which reflected the absence of any parallelism in the prototype and directly informed the job system design of the final engine.

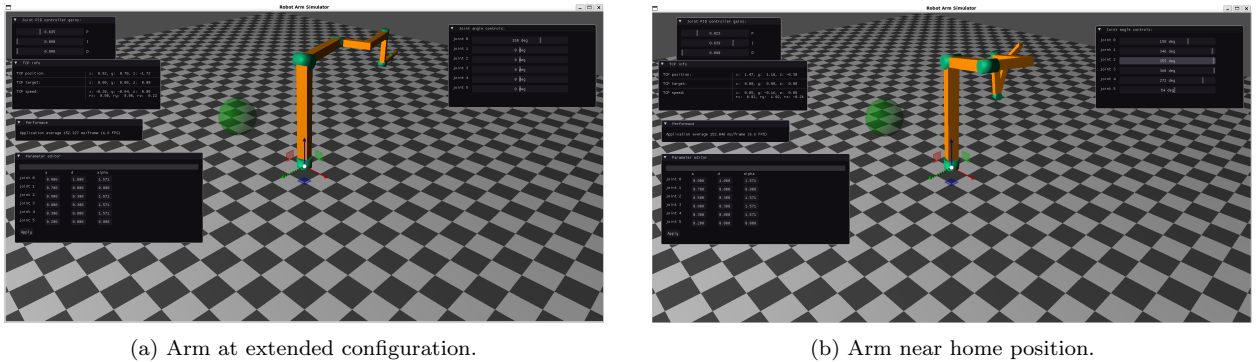


Figure 9.2: Phase 2 Robosim prototype showing DH-parameterised six-joint arm with live TCP telemetry and PID gain controls.

9.1.3 Phase 3: Matrix Transform Visualizer

An interactive 2D matrix transform visualizer was built as a standalone demonstration for linear algebra concepts. Figure 9.3 shows the tool displaying an arrow shape under a rotation of -89.2 degrees. The right panel exposes

rotation, scale (X/Y), and shear (X/Y) as interactive sliders. The resulting 3×3 transformation matrix is displayed numerically alongside the determinant and an area preservation indicator. With the current rotation, the determinant reads 1.000 and the display confirms that area is preserved, which is the expected result for a pure rotation.

The tool supports four shape primitives (Square, Arrow, House, and Letter F) that can be cycled with the Tab key, and includes auto-rotate and reset controls. The original shape is rendered as a ghost outline while the transformed shape is drawn in solid colour, with x-grid and y-grid lines showing how the coordinate system itself deforms under the transformation.

This prototype validates the pedagogical scaffolding approach: instead of asking students to compute matrix entries by hand, the visualizer lets them drag a slider and immediately see how rotation, scale, and shear map to specific matrix values and geometric outcomes. The determinant readout connects the abstract concept of area scaling to a visible quantity that updates in real time.

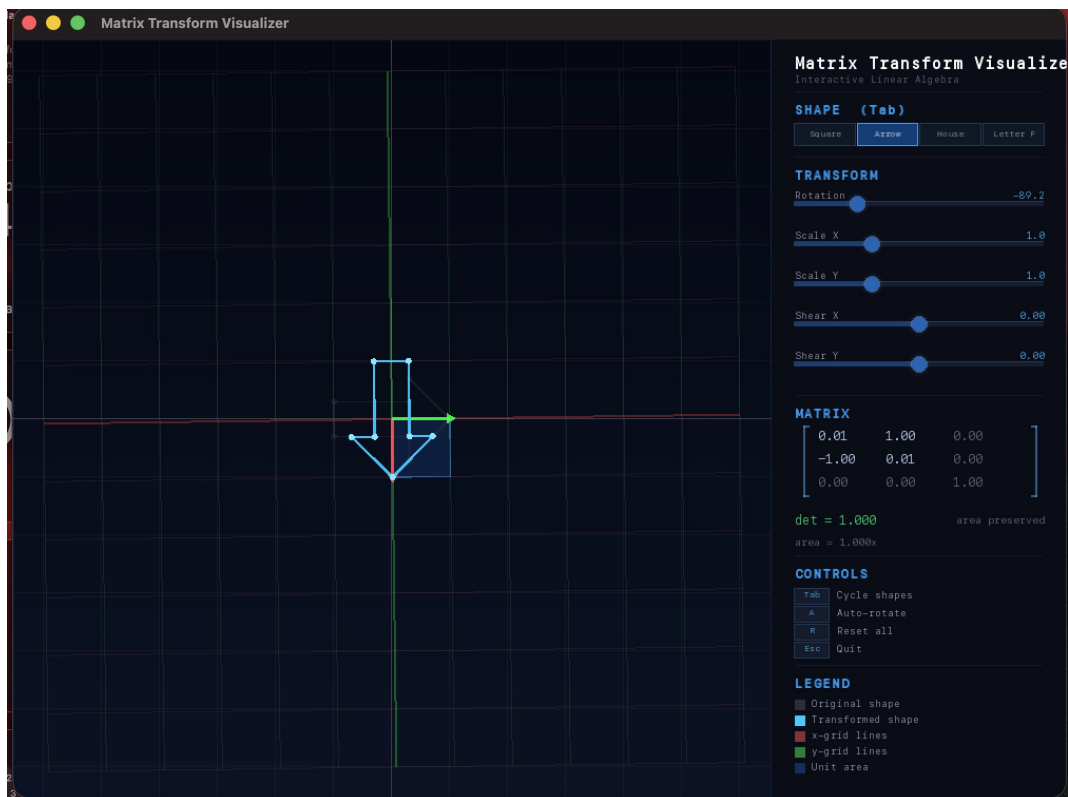


Figure 9.3: Interactive matrix transform visualizer showing an arrow shape under -89.2° rotation with live matrix display, determinant readout, and area preservation indicator.

9.2 Simulation Subsystem Results

9.2.1 XPBD Cloth and Soft Body

The XPBD cloth simulation was evaluated with 4,541 particles and 4 solver iterations. Figures 9.4a–9.4c show three representative scenes: a cloth draped over a sphere, rigid body meshes resting on a cloth platform, and multi-body interaction with the Stanford Bunny and Armadillo models. Frame rates ranged from 24 to 42 FPS depending on the number of active collision constraints. The compliance parameters (**stretch** = 0.0001, **bend** = 0.0001) remained stable across all scenes without requiring timestep adjustment, validating the timestep-invariant property of the XPBD formulation. The cloth-sphere interaction in Figure 9.4a produced 57 active collision constraints simultaneously without instability.

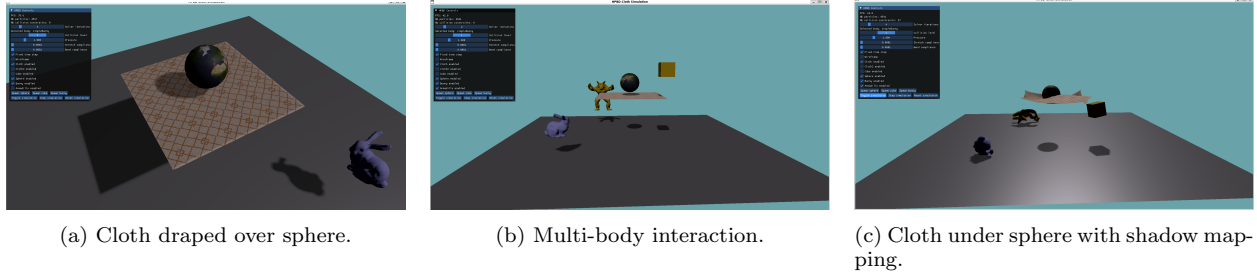


Figure 9.4: XPBD cloth simulation with 4,541 particles and 4 solver iterations across three interaction scenarios.

9.2.2 Mass-Spring Flag Simulation

Figures 9.5a and 9.5b show the cloth flag prototype evaluating sphere collision and wind response. The parameter panel exposed spring constants (K_0 – K_2), damping coefficients (V_0 – V_2), wind velocity, and sphere radius at runtime. The cloth correctly draped around and parted from the sphere as it moved, demonstrating robust contact response under large deformations without constraint deletion.

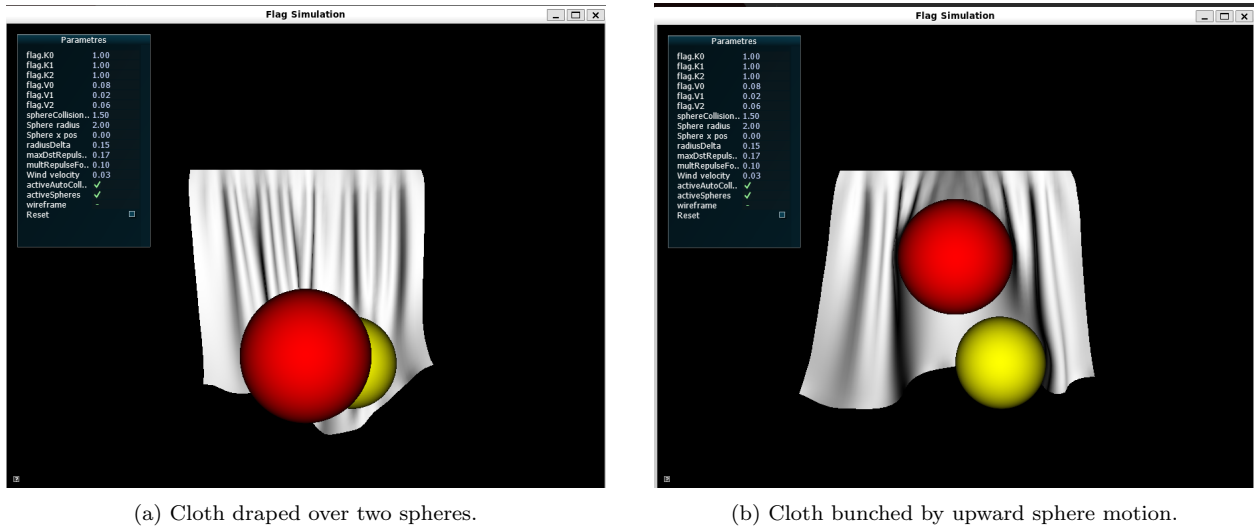


Figure 9.5: Flag cloth prototype showing sphere collision and wind response with runtime parameter exposure.

9.2.3 GPU Particle System

The GPU particle benchmark rendered 100,000 particles on an NVIDIA GTX 1650 using OpenGL 4.2 under Mesa. The terminal output below summarises the pipeline breakdown.

Figure 9.6 shows the particle cloud at runtime. The average frame time of 13.53 ms corresponds to 73.91 FPS over 2,321 frames. Rendering consumed only 0.02 ms because all geometry is resolved to instanced points; the dominant cost is CPU-side data copy at 1.34 ms, suggesting that a persistent mapped buffer strategy would reduce this overhead further.

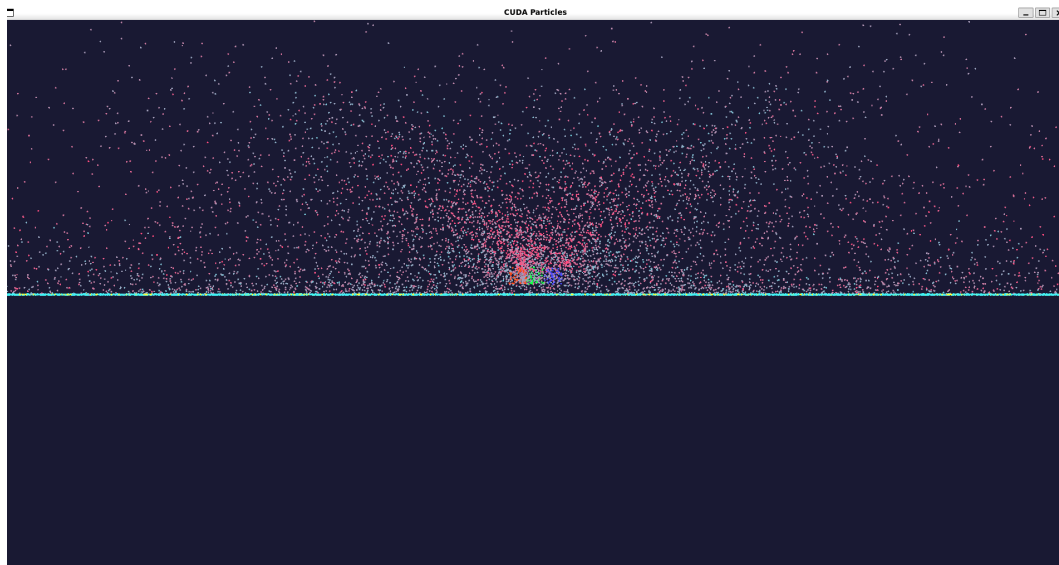


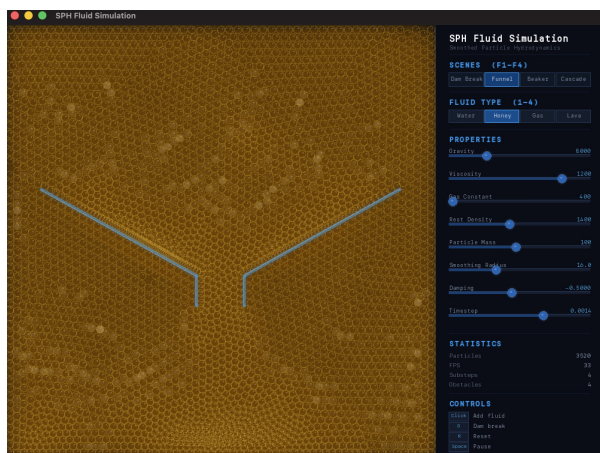
Figure 9.6: GPU particle system rendering 100,000 particles using cuda at 73.91 average FPS on a GTX 1650.

9.2.4 SPH Fluid Demonstration Prototypes

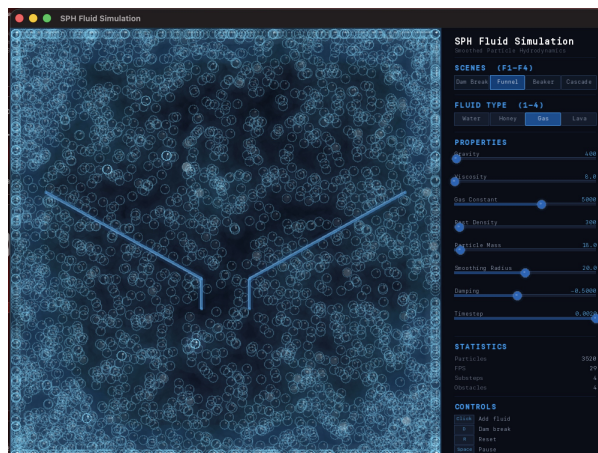
A standalone 2D SPH fluid demonstration was built to validate the fluid simulation pipeline and the parameter-driven authoring workflow independently of the main engine. The demo exposes all SPH parameters (gravity, viscosity, gas constant, rest density, particle mass, smoothing radius, damping, and timestep) through runtime sliders and provides four preset fluid types: Water, Honey, Gas, and Lava. Four scene layouts (Dam Break, Funnel, Beaker, and Cascade) provide different obstacle configurations.

Figures 9.7a–9.8b show the simulation running across all four fluid types. The Honey preset (Figure 9.7a) uses high viscosity (1,200) and high gravity (8,000) to produce a slow, thick flow that clings to the funnel obstacles. With 3,520 particles the simulation held 33 FPS. The Gas preset (Figure 9.7b) drops viscosity to 8.0 and raises the gas constant to 5,000, producing dispersed, fast-moving particles that fill the container volume. Frame rate was 29 FPS at 3,520 particles. The Lava preset (Figure 9.8a) uses the highest gravity (18,000) and high rest density (2,000) with moderate viscosity (800), producing a dense, heavy fluid that pools at the bottom. This was the most expensive configuration at 13 FPS due to the high particle density in the lower region creating more neighbour interactions. The Water preset on the Beaker scene (Figure 9.8b) shows 2,168 particles settling into a container at 27 FPS with moderate gravity (12,000) and viscosity (250).

The key observation across all four presets is that switching fluid types only changes parameter values. The simulation code, the rendering, and the scene layout remain identical. This validates the template-driven authoring model: a teacher can switch between fundamentally different fluid behaviours by clicking a preset button rather than modifying any code or configuration files.

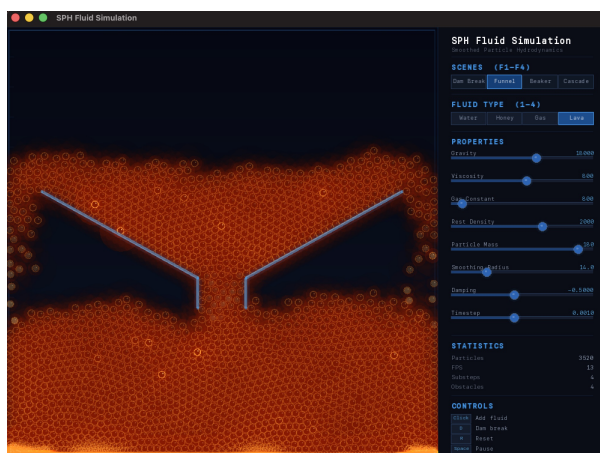


(a) Honey preset: viscosity 1,200, gravity 8,000, 3,520 particles at 33 FPS.

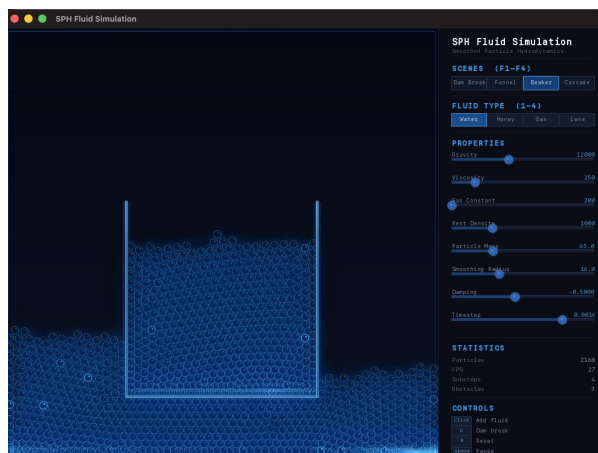


(b) Gas preset: viscosity 8.0, gas constant 5,000, 3,520 particles at 29 FPS.

Figure 9.7: SPH fluid demonstration showing Honey and Gas presets on the Funnel scene with runtime parameter exposure.



(a) Lava preset: gravity 18,000, rest density 2,000, 3,520 particles at 13 FPS.

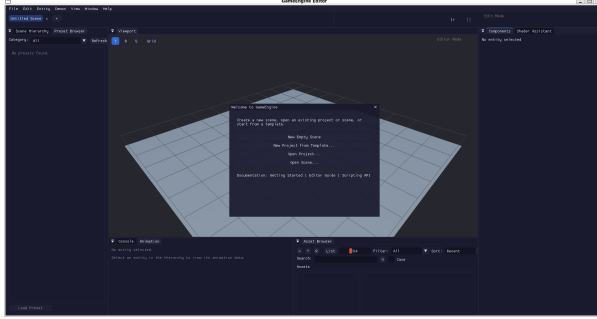


(b) Water preset on Beaker scene: gravity 12,000, viscosity 250, 2,168 particles at 27 FPS.

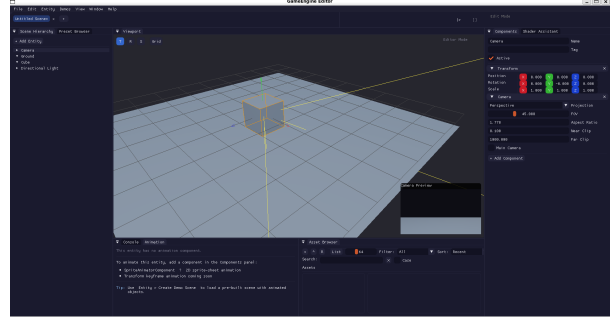
Figure 9.8: SPH fluid demonstration showing Lava and Water presets, illustrating the range of fluid behaviours achievable through parameter changes alone.

9.3 Editor Authoring Layer Results

Figures 9.9a–9.10c document the educator-facing editor across representative workflows. Figure 9.9a shows the WelcomePanel at launch offering template instantiation, project open, and documentation links. Figures 9.9b and 9.10a demonstrate the deferred PBR viewport with point light placement, material parameter editing, and the camera preview inset. Figure 9.10b shows the cloth-with-wind preset loaded from PresetManager, with the scene hierarchy populated automatically. Figure 9.10c shows the rigid body stacking preset with the ComponentsWindow exposing mass, damping, body type, and collider extents without any code interaction.

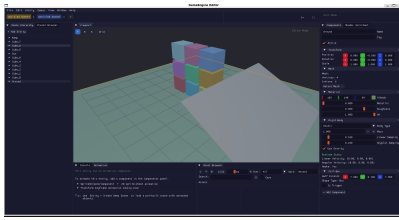


(a) WelcomePanel and template selector at launch.

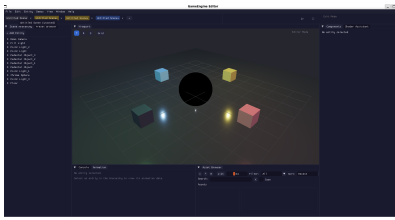


(b) PBR viewport with camera component inspector.

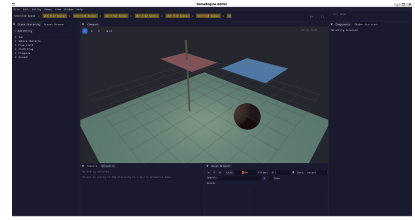
Figure 9.9: Editor startup and scene authoring workflow.



(a) Deferred PBR with point lights.



(b) Cloth preset with hierarchy.



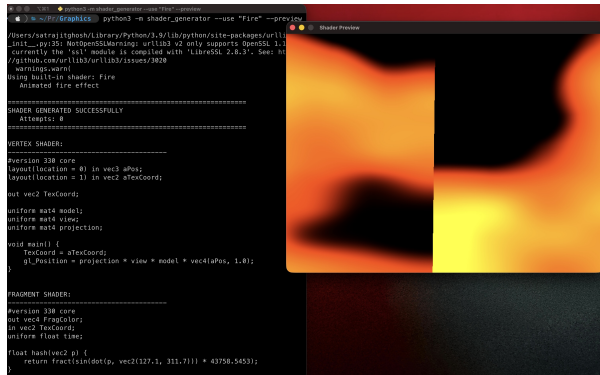
(c) Rigid body stacking with inspector.

Figure 9.10: Editor scenes showing template-instantiated demonstrations with component inspection and no code exposure.

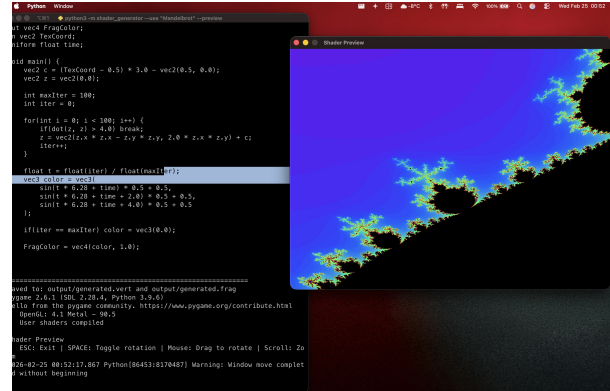
9.3.1 Natural Language Shader Generation

The shader generator was evaluated on two representative prompts to demonstrate the natural language authoring workflow. Given the prompt `--use "Mandelbrot"`, the system produced a complete GLSL fragment shader implementing the Mandelbrot set with animated colour cycling driven by a `time` uniform. Figures 9.11a and 9.11b show the generated source and the live preview respectively. The shader correctly implements the escape-time algorithm with 100 iterations, maps the iteration count to a sinusoidal RGB colour triple, and renders the set boundary with smooth animated colour variation at interactive frame rates under OpenGL 4.1.

Given the prompt `--use "Fire"`, the system generated a noise-based animated fire fragment shader using a hash function and layered smooth-noise octaves to simulate flame movement. Figure 9.12 shows the live preview alongside the generated vertex and fragment shader source. The system reported zero repair attempts for both shaders, indicating that the generated GLSL compiled without errors on the first attempt. Both outputs were saved to `output/generated.vert` and `output/generated.frag` and applied to scene geometry within the engine editor without manual modification.



(a) Generated Mandelbrot GLSL source and animated preview.



(b) Mandelbrot set rendered at full resolution.

Figure 9.11: Natural language shader generation: Mandelbrot and fire prompt producing a compilable animated GLSL shader on the first attempt. These show the noise-based animated fragment shader source alongside the live preview

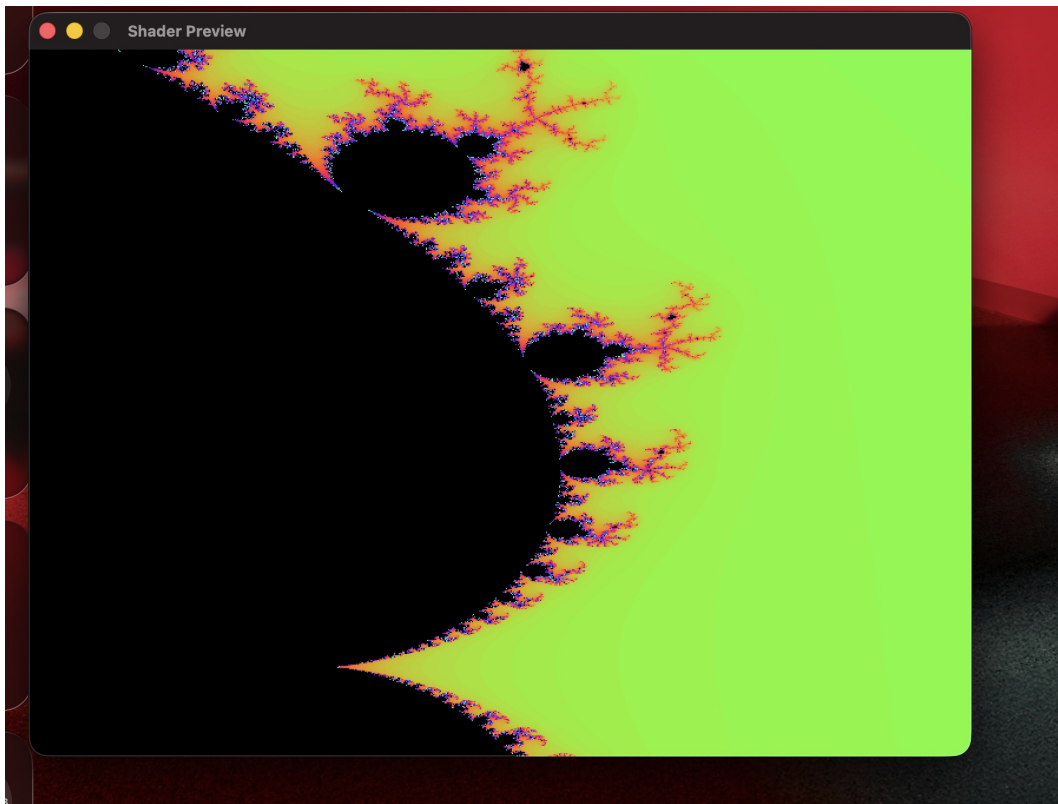


Figure 9.12: Mandelbrot with green background shader generated from the prompt "Mandelbrot with neon green background", showing the live preview. Zero compilation repair attempts were required.

9.3.2 LLM-Assisted CMake Generation

The CMake generator was evaluated using the SPH fluid demonstration project as a test case. The tool uses Qwen2.5-Coder 7B (qwen2.5:7b-instruct-q4_K_M) running locally through Ollama at temperature 0.3 [48, 49].

Figure 9.13a shows the first two steps of the pipeline. In Step 1, the tool analyses the project directory structure,

detecting four source files (`main.cpp`, `sph.cpp`, `renderer.cpp`, `gui.cpp`), four headers, an SFML dependency, C++17 as the language standard, and an executable target type. In Step 2, this information is injected into a structured system prompt that instructs the model to output only valid `CMakeLists.txt` content with no explanations or mark-down, using modern CMake practices including target-based commands, generator expressions, and `find_package` for external dependencies. The total prompt length was 1,340 characters.

Figure 9.13b shows Step 3, where the model streams the generated `CMakeLists.txt` in real time, followed by Step 4 displaying the final 26-line output saved to disk. The generated file correctly set up the project, configured C++17, listed all source files, linked SFML, and included install rules.



(a) Step 1: project analysis and Step 2: structured LLM prompt with injected project specification.

(b) Step 3: Qwen2.5-Coder 7B streaming `CMakeLists.txt` generation and Step 4: final saved output.

Figure 9.13: CMake generation pipeline: project analysis, prompt construction, and LLM-driven output using locally hosted Qwen2.5-Coder 7B.

The initial build attempt failed with a CMake error at `add_executable` (Figure 9.14a). The tool detected the failure and offered to send the error back to the model for automatic repair. Figure 9.14b shows the auto-fix loop: the error message was forwarded to the LLM, which produced a corrected `CMakeLists.txt` adding the missing `target_include_directories` directive and properly closing the `add_executable` block. The fixed file grew from 26 to 30 lines.

After the repair, the build succeeded on the second attempt, compiling across 12 threads, and the SPH fluid demo launched directly from the terminal (Figure 9.15). The full cycle from project analysis through generation, failed build, automatic repair, successful build, and demo launch completed without any manual editing of the `CMakeLists.txt` file.



(a) Step 5: initial build fails; tool offers AI-driven repair.

(b) Auto-fix: error sent to LLM, corrected `CMakeLists.txt` generated with include directory fix.

Figure 9.14: Build failure and automatic LLM-driven repair loop.

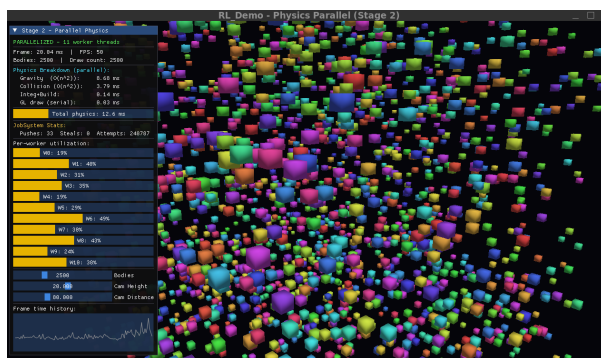


Figure 9.15: Successful build after automatic repair, compiling with 12 threads and launching the SPH fluid demo directly from the terminal.

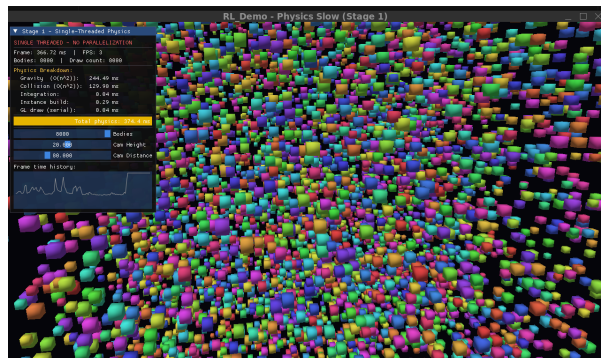
9.4 RL Auto-Tuner Results

9.4.1 Parallelism Baseline

Before evaluating the RL agent, the impact of the job system was established. Figure 9.16a shows single-threaded execution with 8,000 bodies, producing a frame time of 366.72 ms (3 FPS) with gravity and collision each contributing over 100 ms due to their $O(n^2)$ complexity. Figure 9.16b shows the parallel configuration with 2,500 bodies across 11 workers, reducing total physics time to 12.6 ms (50 FPS). Per-worker utilisation ranged from 19% to 49%, indicating workload imbalance that the job scheduler environment was designed to address.



(a) Single-threaded: 8,000 bodies, 366.72 ms, 3 FPS.



(b) Parallelised: 8,000 bodies, 20.04 ms, 50 FPS.

Figure 9.16: Single-threaded versus 11-worker parallel physics execution showing the workload imbalance visible in per-worker utilisation bars.

9.4.2 Frame Scheduler Environment

Table 9.1 presents RL agent results against fixed-count baselines for the `FrameSchedulerEnv`. The PPO agent achieved an average frame time of 9.070 ms, a 23.79% improvement over the best static baseline (10k cubes, 11.901 ms). The agent also reduced the p95 latency from 18.687 ms to 15.269 ms, demonstrating that dynamic scene parameter adjustment produces lower and more consistent frame times than any fixed configuration.

Table 9.1: FrameSchedulerEnv: PPO agent versus fixed-count baselines (100 benchmark steps).

Configuration	Avg (ms)	Min (ms)	Max (ms)	p95 (ms)
1k cubes	12.389	7.754	45.942	18.687
10k cubes	11.901	8.593	22.010	15.248
50k cubes	15.648	12.774	24.233	18.761
100k cubes	20.248	15.419	31.320	24.838
PPO agent	9.070	5.404	29.661	15.269

9.4.3 Job Scheduler Environment

Four experiments evaluated the `JobSchedulerEnv` across varying body counts and workload distributions. Results are summarised in Table 9.2 and detailed in Tables 9.3–9.6.

Table 9.2: JobSchedulerEnv summary: PPO agent versus best baseline per scenario.

Scenario	Best Baseline	Baseline (ms)	RL (ms)	Change
Non-uniform 100k	linear_decrease	9.620	8.634	+10.24%
Uniform 100k	uniform	7.921	8.313	−4.96%
High load 200k	uniform	9.515	10.561	−10.99%
Alternate 100k	complexity_aware	8.376	9.931	−18.58%

Table 9.3: Experiment 2A: Non-uniform workload, 100k cubes, 150 steps.

Strategy	Avg (ms)	Std (ms)	p95 (ms)
uniform	10.680	2.800	17.989
front_heavy	11.198	1.749	14.661
back_heavy	19.015	102.944	14.806
alternating	10.132	2.875	12.794
single_heavy	9.725	2.149	14.501
complexity_aware	10.014	1.494	12.515
linear_decrease	9.620	1.565	12.461
PPO agent	8.634	0.802	10.001

Table 9.4: Experiment 2B: Uniform workload, 100k cubes, 200 steps.

Strategy	Avg (ms)	Std (ms)	p95 (ms)
uniform	7.921	0.505	9.001
front_heavy	8.722	2.401	11.579
back_heavy	8.470	0.943	10.283
alternating	8.873	1.967	11.611
single_heavy	8.827	1.668	11.801
complexity_aware	8.467	1.918	10.652
linear_decrease	9.559	2.274	14.081
PPO agent	8.313	1.171	9.755

Table 9.5: Experiment 2C: High load, 200k cubes, 150 steps.

Strategy	Avg (ms)	Std (ms)	p95 (ms)
uniform	9.515	1.053	11.763
front_heavy	10.114	1.530	13.145
back_heavy	10.005	0.959	11.633
alternating	10.482	1.182	12.686
single_heavy	14.715	10.560	42.390
PPO agent	10.561	1.128	12.549

Table 9.6: Experiment 2D: Alternate run, 100k cubes, 150 steps.

Strategy	Avg (ms)	Std (ms)	p95 (ms)
uniform	19.271	6.840	31.634
front_heavy	17.620	3.643	24.112
back_heavy	13.492	5.215	22.439
alternating	10.616	3.019	16.434
single_heavy	9.214	1.736	11.811
complexity_aware	8.376	1.624	10.855
linear_decrease	8.629	0.935	10.668
PPO agent	9.931	2.137	14.914

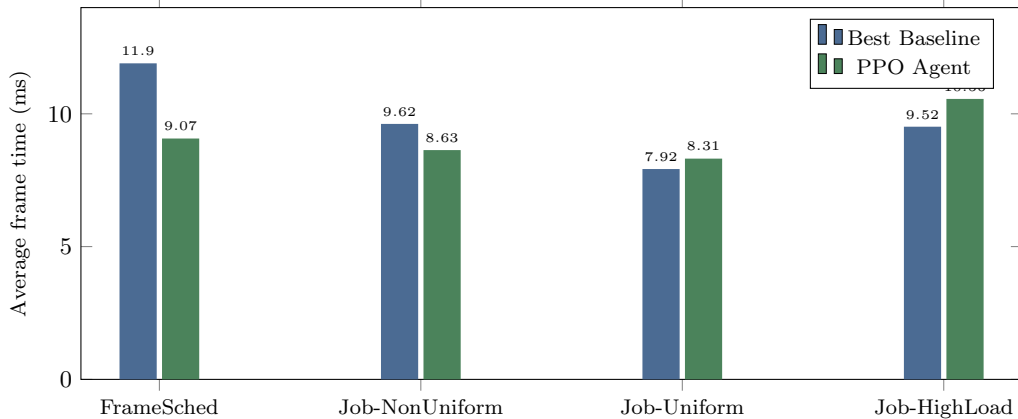


Figure 9.17: PPO agent average frame time versus best static baseline per scenario. Lower is better. The agent outperforms on dynamic and non-uniform workloads; static uniform distribution wins on predictable loads.

Chapter 10

Conclusion and Future Work

10.1 Conclusion

This thesis presented LAGEngine, a unified C++17 engine integrating a custom physics runtime, an educator-facing visual editor, and a passive reinforcement learning auto-tuner within a single architecture. The system was designed to resolve a persistent gap in educational tooling: no existing platform simultaneously addresses accessible classroom authoring, modern physics fidelity, and stable performance on heterogeneous low-power hardware.

The engine core demonstrated that rigid body dynamics, XPBD cloth and soft bodies, and SPH fluid simulation can coexist within a single ECS-driven pipeline without sacrificing the solver transparency required for adaptive control. The XPBD compliance model proved especially important: iteration counts can be reduced under hardware pressure without altering qualitative material behaviour, a property that black-box physics libraries cannot offer. The deferred rendering pipeline, render graph, and AZDO submission strategy produced stable frame costs on OpenGL 4.x hardware representative of constrained classroom environments.

The editor demonstrated that one-click template instantiation, structured parameter panels, and reliable undo reduce time-to-demonstration to minutes for users with no engine programming background. The RL auto-tuner showed a clear operational profile: the PPO agent achieved a 23.79% frame time reduction in the dynamic frame scheduling environment and a 10.24% improvement under non-uniform job distributions, but degraded under perfectly uniform workloads where hand-tuned heuristics are near-optimal. Learned worker weight distributions were directly inspectable, satisfying the explainability requirement for educator-facing deployment.

These results support the central thesis claim that template-driven authoring, a performance-oriented multi-threaded runtime, and a passive RL-based auto-tuner can be integrated into a single platform that is accessible to non-programmer educators while remaining stable on constrained classroom hardware.

10.2 Future Work

Several directions follow naturally from this work. The RL evaluations were limited to short benchmark windows and a single hardware configuration; future work should extend training budgets, apply curriculum learning schedules, and evaluate across a broader range of classroom hardware including integrated GPUs and older driver stacks. A key goal is to experiment with additional algorithms beyond PPO and DDPG, and ultimately to develop a purpose-built

RL algorithm tailored specifically to real-time physics engine tuning, one that encodes domain knowledge about frame time stability and solver behaviour directly into its objective. On the simulation side, the three physics paradigms currently resolve independently and extending the XPBD solver to support cross-paradigm coupling would broaden the range of demonstrations significantly. A Vulkan backend would unlock parallel command recording and expose additional memory parameters to the auto-tuner. A formal usability study with real classroom participants would provide empirical grounding for the accessibility claims made here. Finally, LAGEngine is intended to be released as a fully open-source project, making the engine core, editor, RL environments, and training suite freely available for the research and education communities to build on, extend, and deploy.

Beyond engine internals, one direction that received preliminary exploration during this work is NeRF-based avatar generation. The idea is to let students place photorealistic versions of themselves into running simulations using nothing more than casual phone photographs, removing the need for manual 3D modeling or professional scanning equipment. A collaborative investigation with Thomas Yang [52] produced a working pipeline that covers pose refinement, transformer-based view selection, volumetric reconstruction, mesh extraction, and relighting. The following subsection describes that pipeline, its current results, and the remaining integration work needed to connect it to the LAGEngine runtime.

10.2.1 NeRF-Based Avatar Integration

A persistent limitation of classroom physics demonstrations is that students interact with generic character assets that have no personal connection to them. If a student could place a photorealistic version of themselves into a simulation, walking through a fluid dynamics demo or colliding with rigid bodies, the experience becomes far more engaging. That motivation drove a preliminary investigation into Neural Radiance Field (NeRF)-based avatar generation as a future extension of the engine.

This subsection describes the explored pipeline in detail. The work was conducted as a separate system in collaboration with Thomas Yang [52] and is not yet connected to the engine runtime, but it establishes a concrete architectural path from image capture through reconstruction, mesh and texture generation, and engine-side import.

Motivation and integration goal

The barrier between real users and interactive simulation spaces remains high. Traditional asset-authoring workflows require either manual 3D modeling or carefully controlled multi-view scanning setups, neither of which is realistic in a classroom setting where students and instructors are not 3D artists. A NeRF-driven avatar pipeline offers a more practical alternative: creating personalized digital representations from casually collected photographs captured on an ordinary phone.

Within LAGEngine, the long-term objective is to connect a reconstructed model to the engine’s runtime asset pipeline so that a user can place themselves into a running simulation using lightweight, widely available input data. The early work described here already covers image capture, reconstruction, mesh extraction, relighting, and texture synthesis, which are the pieces most relevant to eventual engine integration.

Pipeline overview

The explored avatar pipeline converts unstructured image collections into animatable 3D avatars through a sequence of differentiable modules. The system does not assume known camera intrinsics or dense viewpoint coverage, which makes it applicable to the casual capture scenario described above.

Figure 10.1 shows the full system architecture. The pipeline proceeds through five stages: (1) preprocessing and metadata extraction, (2) pose and focal length refinement, (3) transformer-based view selection, (4) downstream NeRF reconstruction, and (5) mesh extraction with relighting and texture synthesis.

Unstructured images enter from the left and are processed in three parallel tracks. DeepLabV3 [53] segmentation produces foreground masks. MediaPipe [54] converts raw images to LLFF-format pose estimates. DINOv2 [55]

extracts visual embeddings. The coarse poses are fed into a Gaussian Fourier Feature encoder alongside intrinsic parameters, producing refined rotation, translation, and focal length estimates via a NeRF-supervised rendering loss ($\mathcal{L}_{SSIM}$, $\mathcal{L}_{pix}$). These refined parameters, together with the DINOv2 embeddings, are assembled into 394-dimensional multimodal tokens that enter the MTCM Transformer Module. The MTCM selects the most informative views, which are passed both to a TinyNeRF for selection-loss supervision and to the downstream pipeline for final image prediction.

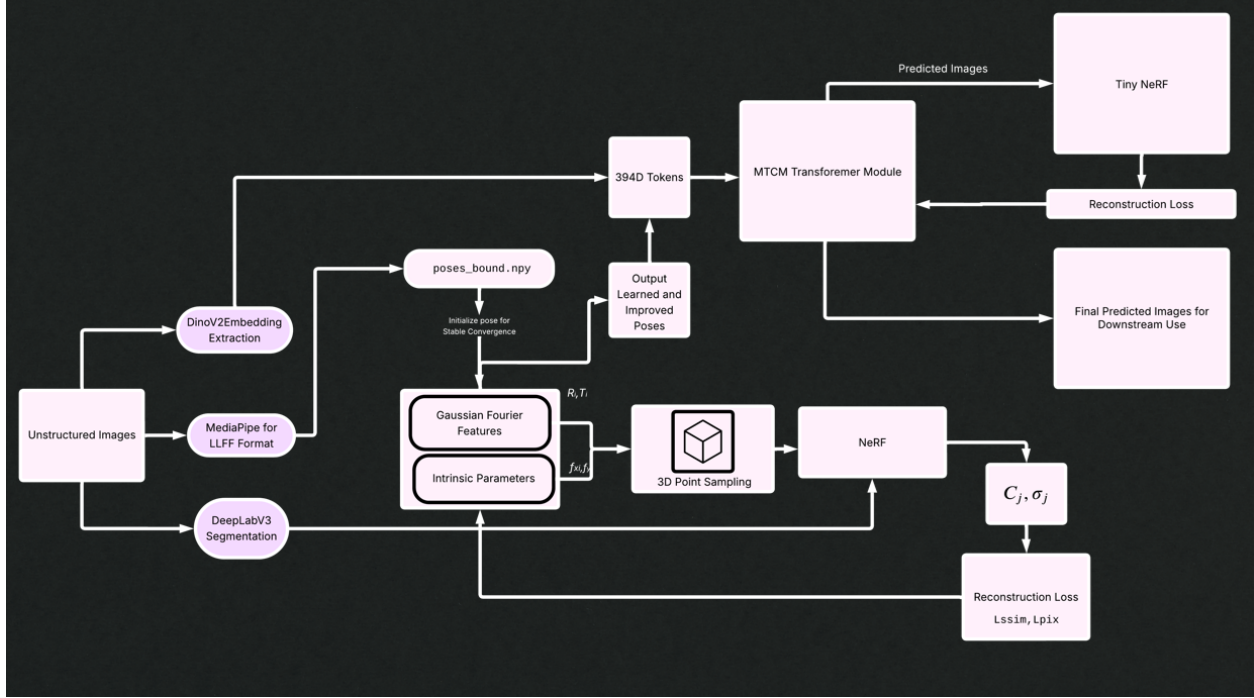


Figure 10.1: System architecture for the integrated pose refinement and view selection pipeline. Unstructured images are preprocessed through segmentation (DeepLabV3), pose estimation (MediaPipe to LLFF format), and embedding extraction (DINOv2). Coarse poses are refined via Gaussian Fourier Features and an MLP-based regressor, supervised by a NeRF rendering loss ($\mathcal{L}_{SSIM}$, $\mathcal{L}_{pix}$). The refined poses and embeddings form 394D multimodal tokens that enter the MTCM Transformer Module, which selects optimal views. A TinyNeRF module provides reconstruction-loss supervision for the selection process, and the final predicted images are passed to the downstream reconstruction pipeline.

Preprocessing

The pipeline begins by converting an unstructured image collection into a structured, LLFF-compatible format enriched with semantic and geometric metadata. This stage is necessary for stable downstream optimization, especially without calibrated multi-view input.

Each image first undergoes semantic segmentation using DeepLabV3 [53] to isolate the foreground subject and generate RGBA masks. These masks improve the robustness of both pose estimation and NeRF supervision by filtering out cluttered backgrounds and occlusions.

Coarse camera extrinsics are then estimated using a hybrid method that combines MediaPipe’s [54] facial landmark detector with OpenCV’s Perspective-n-Point (PnP) solver [56]. Given 2D facial landmarks and a sparse 3D head model, the PnP solver computes initial rotation and translation vectors for each image. These estimates are stored as `poses_bound.npy` files in COLMAP-compatible LLFF format for downstream processing, as visible in the left portion of Figure 10.1.

In parallel, DINOv2 [55] extracts high-dimensional visual embeddings from each masked image. These embeddings capture visual and semantic information that is later fused with pose, resolution, and segmentation metadata to form

the multimodal tokens consumed by the transformer module. By the end of preprocessing, each image is associated with a coarse pose estimate, a foreground mask, and a feature embedding.

Pose and focal length refinement

The coarse poses from MediaPipe are not accurate enough for high-fidelity NeRF reconstruction. To address this, the pipeline adopts a lightweight adaptation of the NeRFtrinsic Four formulation [57] to refine both pose and focal length estimates.

Initial extrinsics from MediaPipe provide rough pose approximations, while focal length priors come from resolution metadata. These are passed into an MLP-based regressor that encodes camera parameters using Gaussian Fourier features. This encoding makes gradient-based learning of rotation vectors R_i , translation vectors T_i , and intrinsic parameters f_x, f_y more stable than direct parameterization. As shown in the center of Figure 10.1, the Gaussian Fourier Features block and Intrinsic Parameters block feed into 3D Point Sampling, which drives the NeRF rendering used for supervision.

This module is supervised via a volumetric rendering loss computed by a NeRF model trained on the predicted poses. The system minimizes a combination of SSIM and pixel-wise reconstruction losses, $\mathcal{L}_{\text{SSIM}} + \mathcal{L}_{\text{pix}}$, which encourages accurate reprojection while regularizing the solution space. The NeRF produces per-point color C_j and density σ_j values that are volume-rendered and compared against ground-truth pixels.

A key modification relative to the original NeRFtrinsic Four architecture is that only the refined pose and focal parameters are extracted during training, rather than relying on NeRFtrinsic’s own volumetric reconstructions. This decoupling reduces computational overhead and allows faster convergence during joint training with the view selection transformer. The output learned and improved poses are then forwarded to the MTCM module for view selection.

Transformer-based view selection (MTCM)

Sparse-photo reconstruction quality depends heavily on which images are selected for training. Most existing NeRF pipelines treat all inputs equally, which leads to degraded results when the input set contains occlusions, extreme poses, or redundant viewpoints. The pipeline addresses this with a Multi-Token Context Model (MTCM) that learns to rank and select optimal views.

The MTCM receives a sequence of multimodal tokens. Each token comprises a 384-dimensional DINOv2 visual embedding concatenated with a 9-dimensional geometric vector containing the 3D translation, 3D rotation (expressed as axis-angle or 6D representation), and 2D focal length. This produces a 393-dimensional token per view, optionally extended with a scalar segmentation mask area and 2D image resolution to reach 394 dimensions. These are the “394D Tokens” shown entering the MTCM Transformer Module in Figure 10.1.

These tokens are normalized and linearly projected into a latent feature space. To inject spatial inductive bias, the MTCM applies both fixed sinusoidal positional encodings and a learned directional encoding block. The directional block processes pose and focal metadata via an MLP and reintegrates the result into the embedding stream, conditioning downstream attention on camera geometry.

The model backbone is a stack of transformer encoder blocks using multi-head self-attention (MHSA). In each block, token embeddings are split into queries Q , keys K , and values V through learned linear projections, divided into h independent heads, and used to compute scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (10.1)$$

where d_k is the dimension of each head. The outputs of all heads are concatenated and projected back to the latent dimension, then passed through a feed-forward MLP with GELU activation and LayerNorm. Each block includes residual connections around both the attention and MLP components.

At the final layer, a selection head maps the token embeddings to scalar logits, normalized via softmax to yield per-view selection weights. During training, the top-weighted views are selected and rendered via a lightweight

TinyNeRF module (top-right of Figure 10.1). The rendering is compared against a held-out target view using a reconstruction loss ($\mathcal{L}_{\text{MSE}}$ or $\mathcal{L}_{\text{SSIM}}$), and gradients propagate through the entire transformer stack. This allows the model to learn view selection strategies that maximize scene renderability, pose diversity, and reconstruction quality.

FDNeRF-based downstream reconstruction

The selected views feed a custom downstream module adapted from FDNeRF [58]. Unlike conventional NeRF pipelines, FDNeRF incorporates dynamic-aware feature warping and attention-guided multi-view aggregation. Both of these properties matter for sparse-view 3D head reconstruction, where fine pose variation and identity preservation are important.

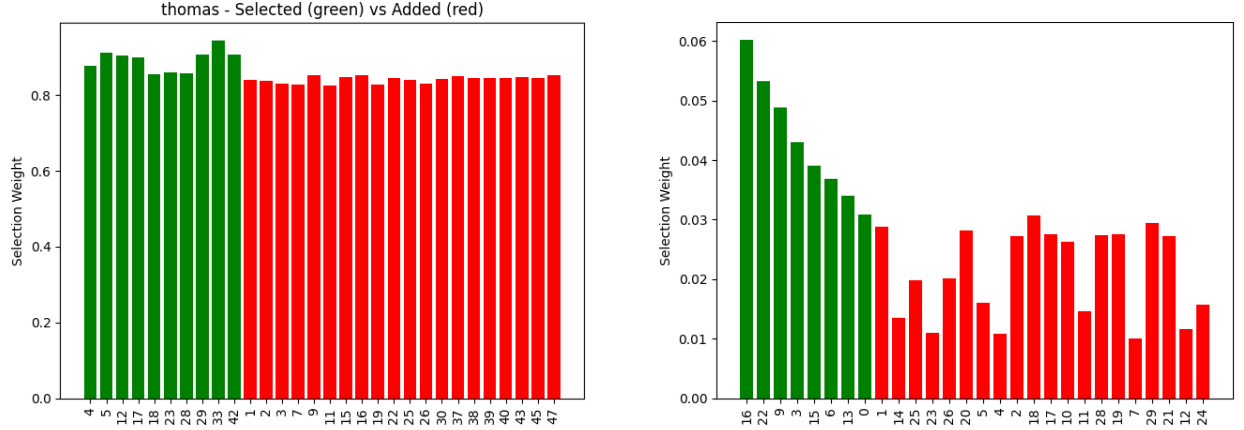
The architecture retains FDNeRF’s CNN encoder structure, augmented with a lightweight attention mechanism that maps query rays from the target view to encoded features from reference views. The attention network processes transposed query and key tensors, reshapes them into a shared latent space, and computes feature relevance via a learnable scoring function. These attention scores modulate the aggregated volume features passed to the NeRF MLP, which improves feature fidelity and multi-view consistency. This allows the system to selectively weight high-quality or spatially consistent observations while suppressing occluded or corrupted regions.

The training follows a two-phase scheme. In the first phase, FDNeRF is pretrained on a large volume of front-facing face data from VoxCeleb [59], which provides a warm start for learning basic facial geometry and appearance. After two to three thousand training images, the second phase switches to data from Meta’s Multiface dataset [60] and the Celebrity Face Dataset [61], which offer more varied viewing angles and lighting conditions. This staged approach speeds up convergence and improves generalizability across identities and lighting variations.

Experimental results

The pipeline was evaluated across multiple datasets, including a personal webcam-captured dataset, celebrity portraits from the Celebrity Face Dataset [61], and subjects from the Meta Multiface dataset [60]. All FDNeRF experiments were conducted on an NVIDIA A100 80GB GPU over approximately 10 hours of training.

View selection and data collection comparison. Figure 10.2 compares MTCM selection weight distributions between webcam-captured data and user-uploaded images for the same subject. The webcam-captured dataset (Figure 10.2a) exhibits tightly clustered, high selection weights, all above 0.85, indicating uniform informativeness and consistent framing across views. Even the non-selected views (red) maintain relatively high weights, reflecting the overall quality of guided capture. In contrast, the uploaded image set (Figure 10.2b) shows a broader spread in selection weights, with the top views reaching about 0.06 and a clear drop-off into a long tail of lower-weighted views. The selected views (green) are more sharply differentiated from the rejected views (red) because the MTCM must work harder to separate useful images from noisy or redundant ones. This difference confirms that guided capture via the custom web interface yields more stable inputs for downstream reconstruction.



(a) Webcam-captured dataset. Selection weights are tightly clustered above 0.85 for all views, with selected views (green) marginally higher than added views (red).

(b) User-uploaded image dataset. Selection weights show a wider spread with a steeper ranking curve between selected (green) and rejected (red) views.

Figure 10.2: Comparison of MTCM selection weights across different data sources for the same subject. (a) Structured webcam-captured data yields uniformly high weights, reflecting consistent quality from guided capture. (b) Unstructured uploaded images produce more differentiated selection patterns, confirming the benefits of guided capture for reliable view selection.

FDNeRF reconstruction quality. Figure 10.3 shows a representative FDNeRF reconstruction result at epoch 2000. The leftmost images are the MTCM-selected input views after segmentation and background removal, showing the subject from a range of angles. The second-to-last image shows the volumetric density field learned by the network, a blue-tinted visualization of the 3D structure the model has internalized. The rightmost image is the final rendered output at the held-out target view, achieving a PSNR of 17.587 dB. The result preserves recognizable identity features such as facial structure, glasses, and hair, despite being reconstructed from only a handful of sparse, casually captured views. The density field visualization confirms that the network has learned a coherent 3D structure rather than simply memorizing 2D appearance.

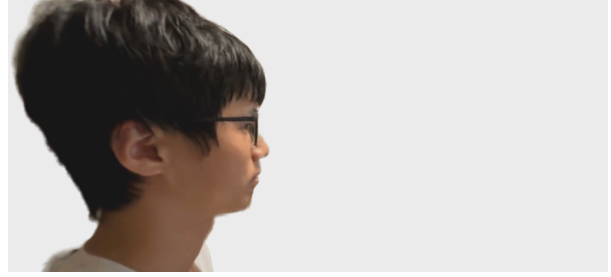


Figure 10.3: FDNeRF reconstruction result at epoch 2000 using MTCM-selected views. From left to right: the selected input views after segmentation and background removal (showing diverse head angles), the learned volumetric density field (blue visualization), and the final rendered output (PSNR: 17.587 dB).

Novel view synthesis. Figure 10.4 shows two novel viewpoints rendered from the same trained model, confirming that the reconstruction generalizes beyond the input views. In the three-quarter view (Figure 10.4a), facial features, glasses, and skin texture are preserved with reasonable fidelity despite this viewpoint not being present in the original input set. The side profile (Figure 10.4b) shows that the ear, jawline, and hair volume remain geometrically consistent, which confirms that the reconstructed radiance field encodes plausible 3D structure rather than view-dependent appearance alone.



(a) Three-quarter view rendering from a novel camera trajectory. Facial features, glasses, and skin texture are preserved despite this viewpoint not being present in the original input set.



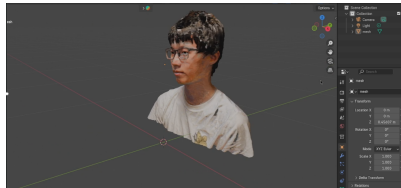
(b) Side-profile rendering under camera motion. The ear, jawline, and hair volume remain geometrically consistent, confirming plausible 3D structure in the reconstructed radiance field.

Figure 10.4: Novel view synthesis results from FDNeRF trained with MTCM-selected views captured through the custom web interface. Both renders show viewpoints not present in the original input images.

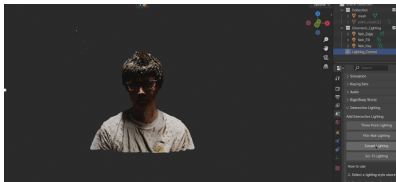
Mesh extraction, relighting, and texture synthesis

3D meshes were extracted from FDNeRF outputs by applying the Marching Cubes algorithm on the predicted signed distance field. RGB textures were applied using UV maps derived from xatlas. These meshes were then passed through a relighting module implemented in Blender. A custom automation toolkit, consisting of Python scripts for camera orbit paths and lighting presets, simulates a range of lighting conditions. The toolkit exposes parameters for lighting mode, HDRI usage, and animation duration, and supports volumetric fog, HDRI environment rotation, and frame-accurate lighting transitions.

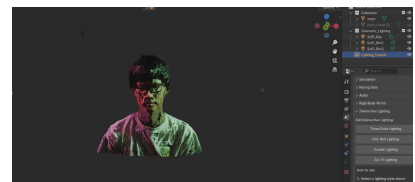
Figure 10.5 shows the full progression from extracted mesh to relit output. The base mesh (Figure 10.5a) is loaded in Blender with the scene outliner showing the mesh object alongside a camera and light. The Transform panel confirms position and scale. The reconstruction captures head shape, hair geometry, glasses, and clothing, though some noise is visible at the top of the head where Marching Cubes produced incomplete surfaces at the volume boundary. Figures 10.5b and 10.5c show the same mesh under two cinematic lighting presets from the custom Interactive Lighting panel. The Film Noir setup uses a three-light rig (Noir_Edge, Noir_Fill, Noir_Key) with deep shadows and high contrast. The Sci-Fi setup uses colored rim lights (SciFi_Key, SciFi_Rim1, SciFi_Rim2), producing a green-and-magenta stylized look. The Interactive Lighting panel, visible in both screenshots, also lists Three Point Lighting and Sunset Lighting as additional one-click presets. That the same mesh responds plausibly to such different lighting conditions suggests sufficient geometric and texture fidelity for downstream rendering tasks.



(a) Textured 3D mesh in Blender after Marching Cubes extraction and xatlas UV unwrapping. The outliner and Transform panel are visible on the right.



(b) Film Noir lighting preset. The Noir key, fill, and edge lights produce high-contrast directional illumination with deep shadows.



(c) Sci-Fi lighting preset with colored rim lights (SciFi_Key, SciFi_Rim1, SciFi_Rim2), producing stylized green-and-magenta illumination.

Figure 10.5: Mesh extraction and relighting results. (a) The base textured mesh extracted via Marching Cubes from FDNeRF's signed distance field. (b, c) The same mesh rendered under two cinematic presets from the scripted Blender lighting controller.

Web-based data collection interface

To reduce pose noise from uncontrolled captures, a custom web-based data collection interface was developed (Figure 10.6). The tool provides two input modes: Webcam Capture for guided multi-pose recording, and Upload Files for existing photographs from smartphones or other cameras. In webcam mode, the interface prompts users to rotate their heads in multiple directions while capturing frames automatically. In both cases, the interface performs segmentation and metadata extraction in real time.

Although the resulting images remain unstructured in the NeRF sense, with no calibrated camera rig, their improved framing and pose diversity lead to higher-quality initial estimates than ad-hoc photo collections. From these captures, the top 8 images are selected by MTCM for final rendering, while the remaining 40 to 50 frames are used to pretrain FDNeRF in the first phase of the two-stage training scheme. The comparison in Figure 10.2 confirms the value of this guided capture approach: webcam-captured data consistently produces higher and more uniform MTCM selection weights than unstructured uploaded images of the same subject.

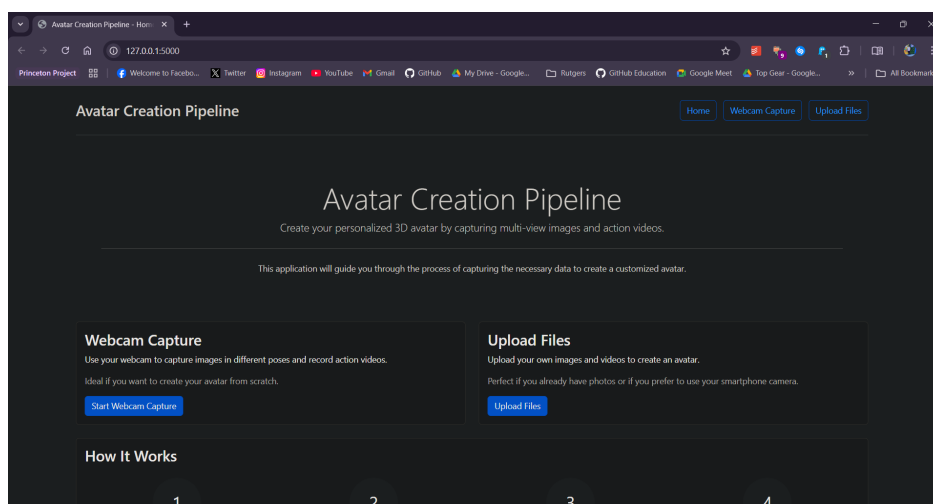


Figure 10.6: The custom-built web interface for image capture and avatar generation. The landing page offers Webcam Capture and Upload Files modes, with a “How It Works” guide below. The interface runs locally and triggers the segmentation and pose prediction pipeline automatically upon submission.

Engine-facing future integration

The next step is to convert this reconstruction pipeline into an engine-facing workflow. In practical terms, that means taking the outputs of the avatar pipeline and mapping them into a representation suitable for runtime use, including geometry, textures, material definitions, and attachment points for animation or interaction. This is where the present gap lies. Reconstruction has been explored, but a robust bridge to import, asset validation, rigging compatibility, and runtime rendering still needs to be built.

A useful future architecture would treat the NeRF system as an offline or semi-offline asset-generation stage, followed by a conversion pipeline that prepares scene-ready outputs for LAGEngine. This would preserve the strengths of neural reconstruction without forcing the core engine runtime to depend on an expensive neural rendering loop. It would also make it easier to expose this feature through the editor in a way that remains practical for classroom use, so that a student can generate their own avatar from a short image capture session and drop it into a running simulation with minimal friction.

Bibliography

- [1] Y. An, L. Haynes, A. D’Alba, and F. Chumney, “Using educational computer games in the classroom: Science teachers’ experiences, attitudes, perceptions, concerns, and support needs,” *Contemporary Issues in Technology and Teacher Education*,

- vol. 16, no. 4, pp. 415–433, 2016. [Online]. Available: <https://citejournal.org/volume-16/issue-4-16/science/using-educational-computer-games-in-the-classroom-science-teachers-experiences-attitudes-perceptions-concerns-and-support-needs/>
- [2] P. Kaimara, E. Fokides, A. Oikonomou, and I. Deliyannis, “Potential barriers to the implementation of digital game-based learning in the classroom: Pre-service teachers’ views,” *Technology, Knowledge and Learning*, vol. 26, no. 4, pp. 825–844, 2021. [Online]. Available: <https://doi.org/10.1007/s10758-021-09512-7>
 - [3] F. Esquembre, “Easy java simulations: A software tool to create scientific simulations in java,” *Computer Physics Communications*, vol. 156, no. 2, pp. 199–204, 2004. [Online]. Available: [https://doi.org/10.1016/S0010-4655\(03\)00440-5](https://doi.org/10.1016/S0010-4655(03)00440-5)
 - [4] Y. Ben-Zion, T. K. Carroll, C. G. West, J. Wong, and N. D. Finkelstein, “Leveraging generative artificial intelligence for simulation-based physics experiments: A new approach to virtual learning about the real world,” *arXiv preprint arXiv:2509.22929*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.22929>
 - [5] C. E. Wieman, K. K. Perkins, and W. K. Adams, “Oersted medal lecture 2007: Interactive simulations for teaching physics,” *American Journal of Physics*, vol. 76, no. 4, pp. 393–399, 2008. [Online]. Available: <https://doi.org/10.1119/1.2815367>
 - [6] N. Rutten, W. R. van Joolingen, and J. T. van der Veen, “The learning effects of computer simulations in science education,” *Computers & Education*, vol. 58, no. 1, pp. 136–153, 2012. [Online]. Available: <https://doi.org/10.1016/j.compedu.2011.07.017>
 - [7] D. Baraff and A. Witkin, “Large steps in cloth simulation,” in *Proceedings of ACM SIGGRAPH*, 1998, pp. 43–54. [Online]. Available: <https://doi.org/10.1145/280814.280821>
 - [8] M. Müller, B. Heidelberger, M. Hennix, and J. Ratcliff, “Position based dynamics,” *Journal of Visual Communication and Image Representation*, vol. 18, no. 2, pp. 109–118, 2007. [Online]. Available: <https://doi.org/10.1016/j.jvcir.2007.01.005>
 - [9] M. Macklin, M. Müller, N. Chentanez, and T.-Y. Kim, “Xpbd: Position-based simulation of compliant constrained dynamics,” in *Proceedings of the 9th International Conference on Motion in Games (MIG)*, 2016. [Online]. Available: <https://doi.org/10.1145/2994258.2994272>
 - [10] M. Müller, D. Charypar, and M. Gross, “Particle-based fluid simulation for interactive applications,” in *Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA)*, 2003, pp. 154–159. [Online]. Available: <https://doi.org/10.2312/SCA03/154-159>
 - [11] M. Ihmsen, N. Akinci, G. Akinci, and M. Teschner, “Sph fluids in computer graphics,” in *Eurographics 2014 – State of the Art Reports*, 2014. [Online]. Available: <https://doi.org/10.2312/egst.20141034>
 - [12] D. Baraff, “Fast contact force computation for nonpenetrating rigid bodies,” in *Proceedings of ACM SIGGRAPH*, 1994, pp. 23–34. [Online]. Available: <https://doi.org/10.1145/192161.192236>
 - [13] J. Gregory, *Game Engine Architecture*, 3rd ed. CRC Press, 2018.
 - [14] R. L. Cook and K. E. Torrance, “A reflectance model for computer graphics,” *ACM Transactions on Graphics*, vol. 1, no. 1, pp. 7–24, 1982. [Online]. Available: <https://doi.org/10.1145/357290.357293>
 - [15] J. T. Kajiya, “The rendering equation,” in *Proceedings of ACM SIGGRAPH*, 1986, pp. 143–150. [Online]. Available: <https://doi.org/10.1145/15922.15902>
 - [16] B. Walter, S. R. Marschner, H. Li, and K. E. Torrance, “Microfacet models for refraction through rough surfaces,” in *Proceedings of the Eurographics Symposium on Rendering (EGSR)*, 2007. [Online]. Available: <https://doi.org/10.2312/EGWR/EGSR07/195-206>
 - [17] P. Debevec, “Rendering synthetic objects into real scenes: Bridging traditional and image-based graphics with global illumination and high dynamic range photography,” in *Proceedings of ACM SIGGRAPH*, 1998, pp. 189–198. [Online]. Available: <https://doi.org/10.1145/280814.280864>
 - [18] V. Kajiya, “Screen space ambient occlusion,” In Wolfgang Engel (Ed.), *ShaderX7: Advanced Rendering Techniques*, Charles River Media, 2009, chapter 2.5. Original real-time SSAO technique developed at Crytek.
 - [19] Y. O’Donnell, “Framegraph: Extensible rendering architecture in frostbite,” Game Developers Conference (GDC), San Francisco, CA, 2017, accessed 2026-02-12. [Online]. Available: <https://www.gdcvault.com/play/1024612/FrameGraph-Extensible-Rendering-Architecture-in>
 - [20] Khronos OpenGL ARB, “GL_arb_buffer_storage extension specification,” 2013, accessed 2026-02-12. [Online]. Available: https://registry.khronos.org/OpenGL/extensions/ARB/ARB_buffer_storage.txt
 - [21] Khronos OpenGL ARB Inc., “Glb multi draw indirect extension specification,” 2012, accessed 2026-02-12. [Online]. Available: https://registry.khronos.org/OpenGL/extensions/ARB/ARB_multi_draw_indirect.txt
 - [22] Khronos OpenGL, “Gl arb parallel shader compile extension specification,” 2017, accessed 2026-02-12. [Online]. Available: https://registry.khronos.org/OpenGL/extensions/ARB/ARB_parallel_shader_compile.txt
 - [23] Khronos Group, “Opengl efficiency: Approaching zero driver overhead,” Game Developers Conference (GDC) Presentation, 2014, accessed 2026-02-12. [Online]. Available: https://www.khronos.org/assets/uploads/developers/library/2014-gdc/Khronos-OpenGL_Efficiency-GDC_Mar14.pdf
 - [24] Microsoft, “wglmakecurrent function (wingdi.h),” Microsoft Learn (Win32 OpenGL), 2022, accessed 2026-02-12. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/wingdi/nf-wingdi-wglmakecurrent>
 - [25] Khronos Group, “Vulkan specification: Threading behavior,” Vulkan 1.3 Specification, 2026, accessed 2026-02-12. [Online]. Available: <https://registry.khronos.org/vulkan/specs/1.3-extensions/html/chap4.html#fundamentals-threadingbehavior>

- [26] Microsoft, “Direct3d 12 cpu efficiency,” DirectX Specifications, 2026, accessed 2026-02-12. [Online]. Available: <https://microsoft.github.io/DirectX-Specs/d3d/CPUefficiency.html>
- [27] Apple Inc., “Metal: Command organization and execution model,” Apple Developer Documentation Archive, 2016, accessed 2026-02-12. [Online]. Available: <https://developer.apple.com/library/archive/documentation/Miscellaneous/Conceptual/MetalProgrammingGuide/CommandOrganizationandExecution/CommandOrganizationandExecution.html>
- [28] Intel GameTechDev, “Presentmon: Etw-based frame presentation capture tool,” GitHub Repository, 2026, accessed 2026-02-12. [Online]. Available: <https://github.com/GameTechDev/PresentMon>
- [29] N. S. Arora, R. D. Blumofe, and C. G. Plaxton, “Thread scheduling for multiprogrammed multiprocessors,” in *Proceedings of the Tenth Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA)*, 1998, pp. 119–129. [Online]. Available: <https://doi.org/10.1145/277651.277678>
- [30] D. Chase and Y. Lev, “Dynamic circular work-stealing deque,” in *Proceedings of the Seventeenth Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA)*, 2005, pp. 21–28. [Online]. Available: <https://doi.org/10.1145/1073970.1073974>
- [31] Epic Games, “Threaded rendering in unreal engine,” Unreal Engine Online Documentation, 2026, accessed 2026-02-12. [Online]. Available: <https://dev.epicgames.com/documentation/en-us/unreal-engine/threaded-rendering-in-unreal-engine>
- [32] Epic Games Documentation, “Parallel rendering overview for unreal engine,” Unreal Engine Online Documentation, 2026, accessed 2026-02-12. [Online]. Available: <https://dev.epicgames.com/documentation/en-us/unreal-engine/parallel-rendering-overview-for-unreal-engine>
- [33] Unity Technologies, “Jobhandle and dependencies,” Unity Manual, 2020, accessed 2026-02-12. [Online]. Available: <https://docs.unity3d.com/2020.1/Documentation/Manual/JobSystemJobDependencies.html>
- [34] Unity Technologies Inc., “Unity.jobs.jobhandle.complete (scripting api),” Unity Scripting API Reference, 2025, accessed 2026-02-12. [Online]. Available: <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Unity.Jobs.JobHandle.Complete.html>
- [35] Godot Engine, “Thread-safe apis,” Godot Engine Documentation, 2026, accessed 2026-02-12. [Online]. Available: https://docs.godotengine.org/en/latest/tutorials/performance/thread_safe_apis.html
- [36] D. Van Aken, A. Pavlo, G. J. Gordon, and B. Zhang, “Automatic database management system tuning through large-scale machine learning,” in *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, 2017, pp. 1009–1024. [Online]. Available: <https://doi.org/10.1145/3035918.3064029>
- [37] J. Zhang, Y. Liu, K. Zhou, G. Li, Z. Xiao, B. Cheng, J. Xing, Y. Wang, T. Cheng, and H. Liu, “An end-to-end automatic cloud database tuning system using deep reinforcement learning,” in *Proceedings of the 2019 ACM International Conference on Management of Data (SIGMOD)*, 2019, pp. 415–432. [Online]. Available: <https://doi.org/10.1145/3299869.3300085>
- [38] R. Li, Y. Chen, C. Tian, and X. Liu, “Athena: Automatic configuration tuning for distributed databases over heterogeneous hardware,” in *Proceedings of the 2023 IEEE International Conference on Data Engineering (ICDE)*, 2023, verify full author list, page numbers, and DOI before submission.
- [39] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [40] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” *arXiv preprint arXiv:1509.02971*, 2015. [Online]. Available: <https://arxiv.org/abs/1509.02971>
- [41] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” *arXiv preprint arXiv:1705.10528*, 2017. [Online]. Available: <https://arxiv.org/abs/1705.10528>
- [42] J. García and F. Fernández, “A comprehensive survey on safe reinforcement learning,” *Journal of Machine Learning Research*, vol. 16, no. 42, pp. 1437–1480, 2015. [Online]. Available: <https://www.jmlr.org/papers/v16/garcia15a.html>
- [43] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” *arXiv preprint arXiv:1709.06560*, 2017. [Online]. Available: <https://arxiv.org/abs/1709.06560>
- [44] J. Denavit and R. S. Hartenberg, “A kinematic notation for lower-pair mechanisms based on matrices,” *Journal of Applied Mechanics*, vol. 22, pp. 215–221, 1955, doi not consistently indexed in older ASME archives; verify before submission.
- [45] E. Coumans, “Bullet physics simulation,” ACM SIGGRAPH 2015 Courses, 2015, verify exact SIGGRAPH course citation against archived proceedings before submission. [Online]. Available: <https://pybullet.org>
- [46] S. R. Buss, “Introduction to inverse kinematics with jacobian transpose, pseudoinverse and damped least squares methods,” University of California, San Diego, Tech. Rep., 2004. [Online]. Available: <https://mathweb.ucsd.edu/~sbuss/ResearchWeb/ikm%20methods/iksurvey.pdf>
- [47] Unity Technologies, “Unity editor interface and layout,” <https://docs.unity3d.com/Manual/LearningtheInterface.html>, 2024, default layout: Hierarchy, Scene view, Inspector, Project, Console. Accessed 2025.
- [48] Ollama, “Ollama: Run large language models locally,” <https://github.com/ollama/ollama>, 2024, open-source local LLM inference framework. Accessed 2025.
- [49] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, “Qwen2.5-coder technical report,” *arXiv preprint arXiv:2409.12186*, 2024.

- [50] Anysphere, Inc., “Cursor: The ai-first code editor,” <https://cursor.com>, 2024, aI-powered IDE built on VS Code with LLM-integrated code generation and editing. Accessed 2025.
- [51] Anthropic, “Claude code: Agentic coding tool,” <https://docs.anthropic.com/en/docs/claude-code>, 2025, terminal-native AI coding agent. Accessed 2025.
- [52] S. Ghosh and T. Yang, “Neural character generation,” Princeton University, Tech. Rep., 2025, unpublished report.
- [53] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, “Deeplabv3: Semantic image segmentation with deep convolutional nets,” 2018, accessed: 2025-05-09.
- [54] Google Research, “MediaPipe: Cross-platform ML solutions for live and streaming media,” 2023, accessed: 2025-05-09.
- [55] Meta AI Research, “DINOv2: State-of-the-art self-supervised vision transformer models,” 2023, accessed: 2025-05-09.
- [56] OpenCV Contributors, “solvePnP — OpenCV documentation,” 2024, accessed: 2025-05-09.
- [57] Y. Lin, M. Chai, Y. Liu, Y. Bao, M. He, D. Lischinski, D. Cohen-Or, and H. Su, “NeRFtrinsic Four: An end-to-end trainable NeRF jointly optimizing scene representation and camera parameters,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2023, pp. 14 534–14 544.
- [58] J. Zhang, X. Li, Z. Wan *et al.*, “FDNeRF: Few-shot dynamic neural radiance fields for face reconstruction and expression editing,” *arXiv preprint*, 2022.
- [59] A. Nagrani, J. S. Chung, and A. Zisserman, “VoxCeleb: A large-scale speaker identification dataset,” *arXiv e-prints*, 2017.
- [60] C.-h. Wu, N. Zheng, E. Brockmeyer *et al.*, “Multiface: A dataset for neural face rendering,” in *arXiv preprint*, 2022.
- [61] Vishesh, “Celebrity face image dataset,” 2022, accessed 2025.